Lecture 8: Clustering, Gaussian mixtures

TTIC 31220: Unsupervised Learning and Data Analysis

Instructor: Karen Livescu

TTI-Chicago

April 15, 2026

Questions re HW2

Questions re HW2

Can we use external pre-trained representations (e.g., learned on ImageNet dataset)? No.

- All work on this homework should use the provided training data only.

Questions re HW2

Can we use external pre-trained representations (e.g., learned on ImageNet dataset)? No.

- All work on this homework should use the provided training data only.

Can we fine-tune the autoencoder? No.

- Common setup: Pre-train + fine-tune
  - Train AE, then add some layers on top of encoder to form the classifier
  - Initialize the "encoder layers" of the classifier with the pre-trained parameters
  - Train full classifier, including the encoder layers, on the labeled data
- On HW2, we don't do this. We learn an AE, freeze the parameters, and use the encoder's output as input to downstream task model (classifier)

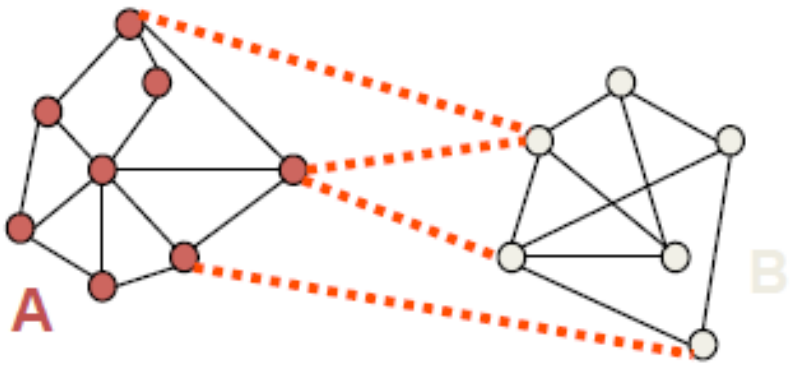
Plan for today

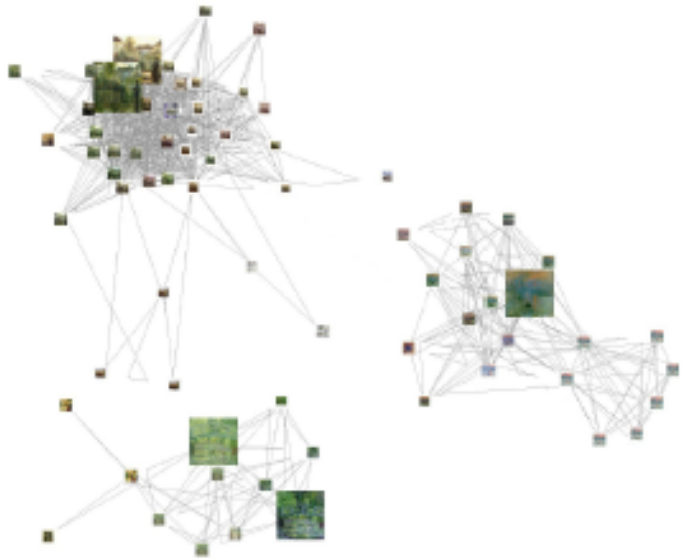
- Spectral clustering
- From clustering to density estimation: Gaussian mixtures
- Other latent variable density models?

Spectral clustering

Spectral clustering

- Use a similarity graph (as in graph-based dimensionality reduction)
- Use spectral (eigenvector-based) methods to divide the graph into connected sub-graphs

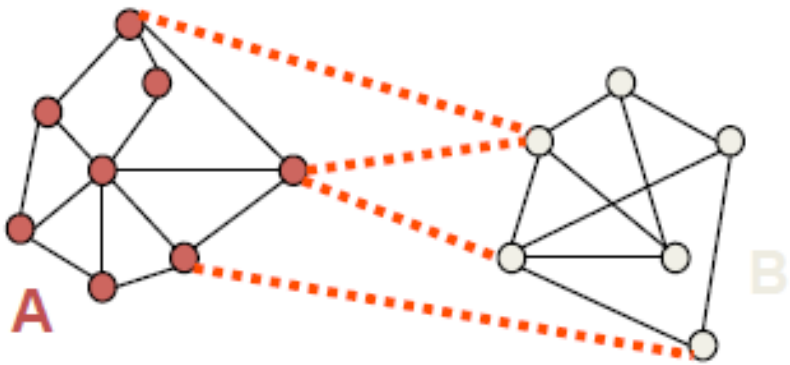


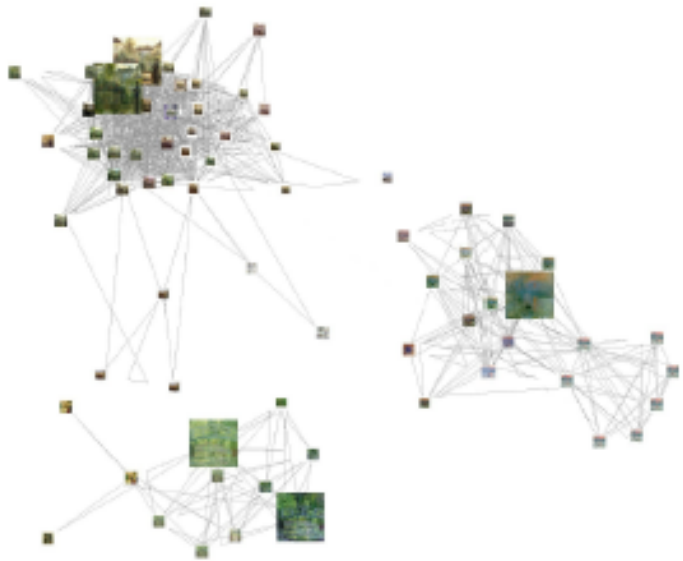


[D. Sontag]

Spectral clustering: preliminaries

- Data $\mathbf{x}_1, \ldots, \mathbf{x}_N$
- Similarities s_{ij}
- Create a graph $G = (V, E)$
  - One node v_i for each $\mathbf{x}_i$
  - Edge weights w_{ij} based on s_{ij}



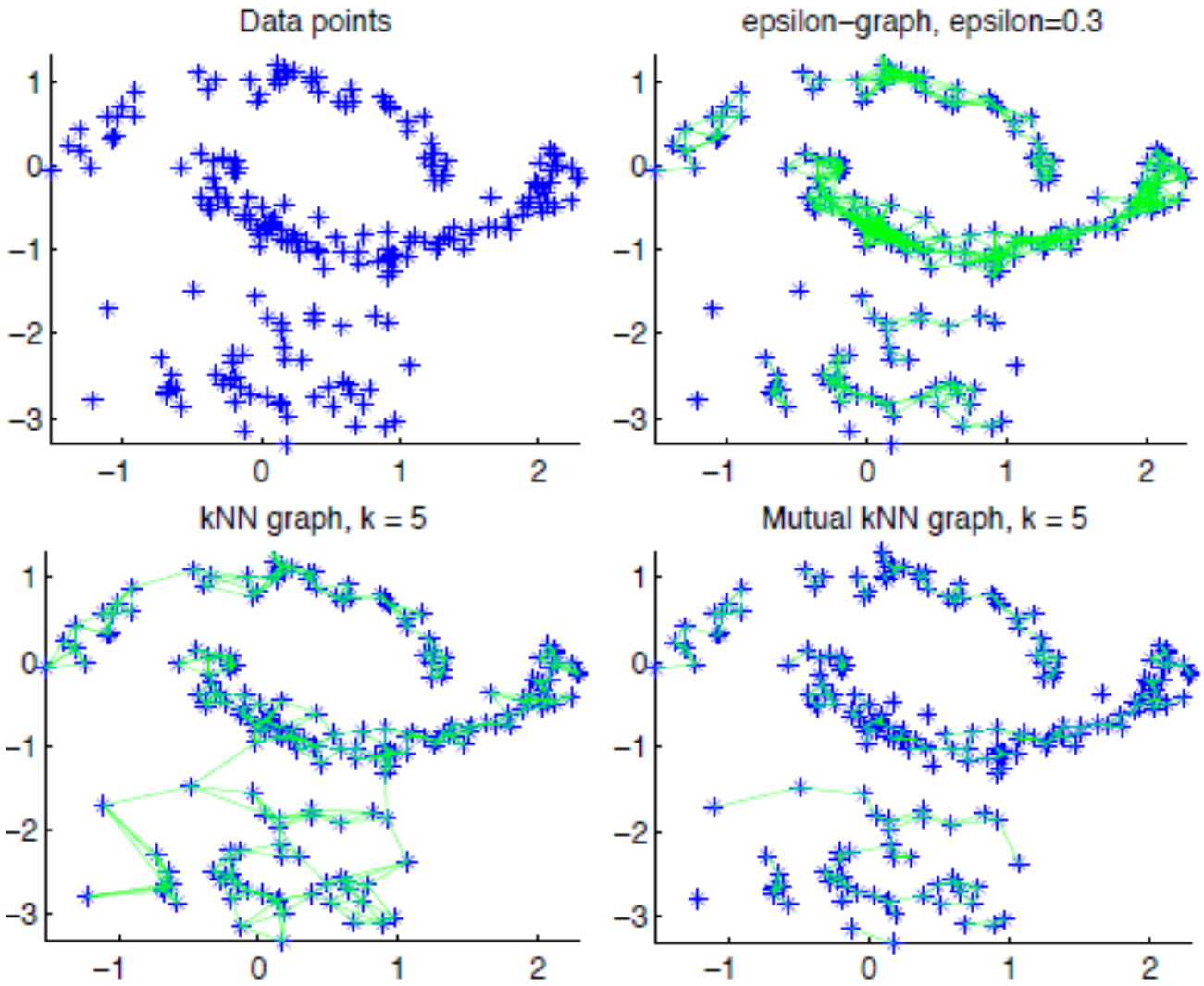


[D. Sontag]

Types of graphs

- Fully connected, weighted by similarity w_{ij}
- ϵ-neighborhood graph: unweighted, include edges with distance $< \epsilon$
- k-NN: include edges where v_i is a k-NN of v_j or vice versa
- Mutual k-NN: include edges where v_i is a k-NN of v_j and vice versa

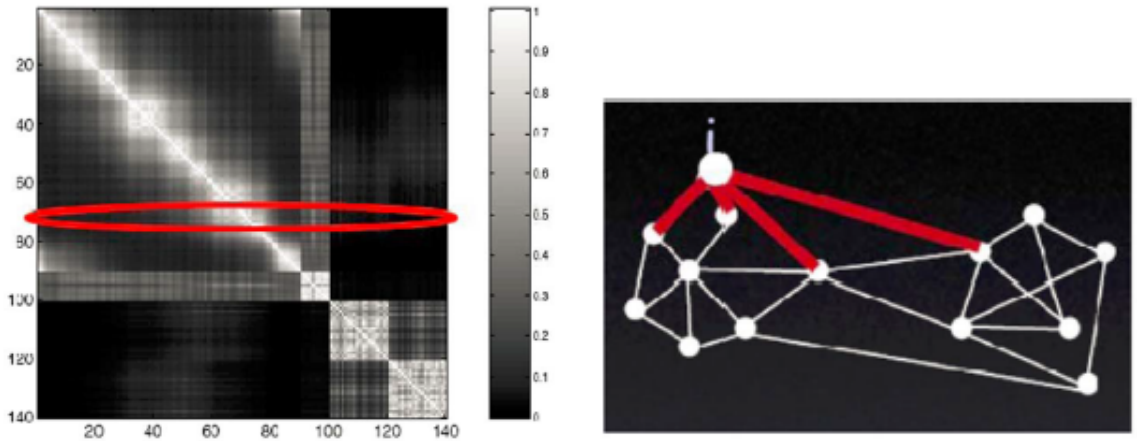
Types of graphs



[von Luxburg]

Adjacency matrices

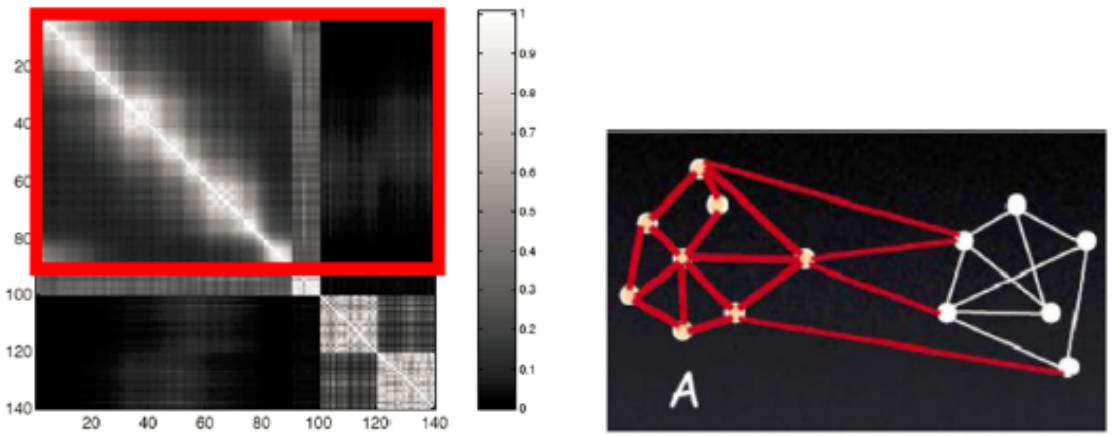
- Adjacency matrix = matrix of edge weights (similarities)
- Can derive quantities of interest from it, e.g.
- Node degree $d_i = \sum_j w_{ij}$



[D. Sontag]

Adjacency matrices

- Adjacency matrix = matrix of edge weights (similarities)
- Can derive quantities of interest from it, e.g.
- Volume of set of nodes $vol(A) = \sum_{i \in A} d_i$

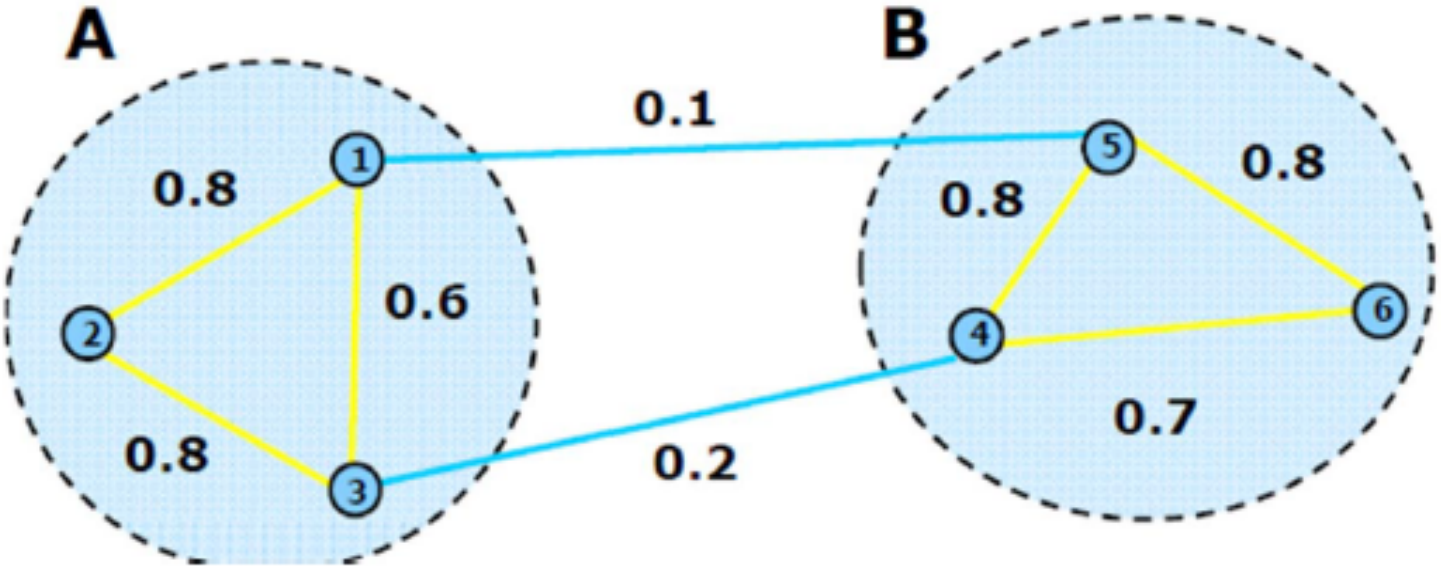


[D. Sontag]

Graph cuts

- Consider some partition of the graph into two sub-graphs **A** and **B**
- $Cut(A, B) = W(A, B) = \sum_{i \in A, j \in B} w_{ij}$ (= 0.3 in this case)
- We would like to minimize the cuts between each cluster and the rest of the graph:

$$\underset{A_1,\ldots,A_k}{\operatorname{argmin}} \frac{1}{2} \sum_{i=1}^{k} W(A_i, \bar{A}_i)$$



[Hamad & Biela]

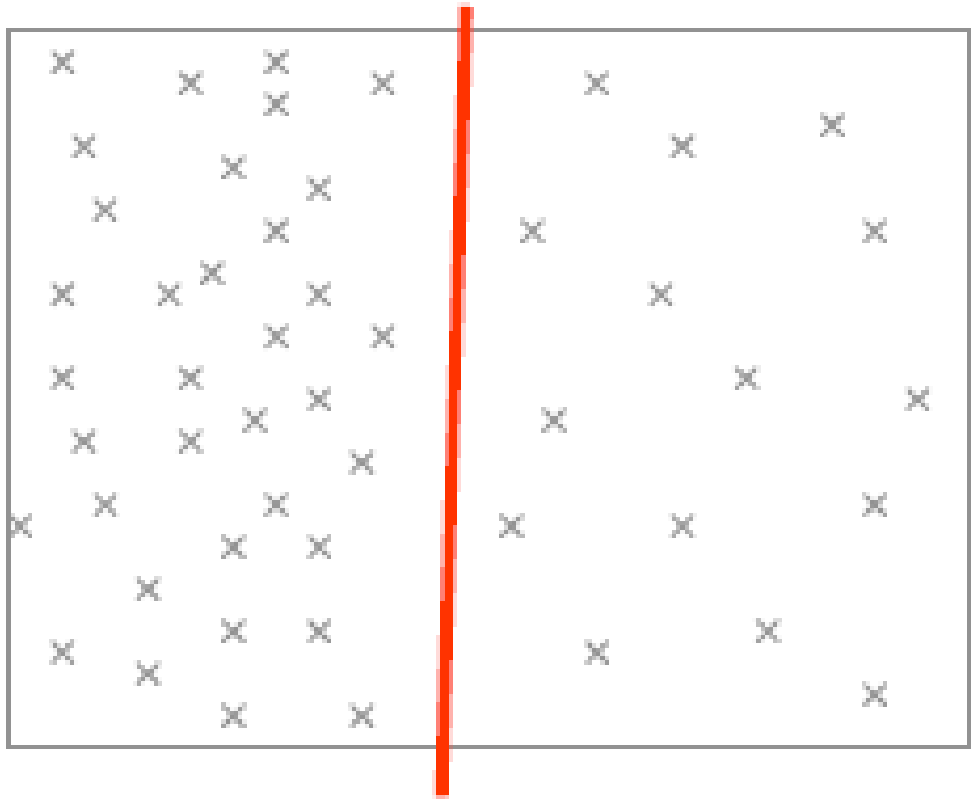
MinCut favors isolated nodes



[E. Fox]

Cuts that account for cluster size

- Ratio cuts (RatioCut) normalizes by cardinality: $\frac{1}{2}\sum_{i=1}^{k}\frac{W(A_i,\bar{A}_i)}{|A_i|}$
- Normalized cuts (Ncut) normalizes by volume: $\frac{1}{2}\sum_{i=1}^{k}\frac{W(A_i,\bar{A}_i)}{vol(A_i)}$



[E. Fox]

Graph Laplacian

Solution to normalized cut is found via a graph Laplacian

- This term has multiple meanings in the literature...
- Let W be the similarity matrix
- Let D be the diagonal degree matrix $D_{ii} = \sum_j W_{ij}$
- Then $L = D - W$ is the unnormalized graph Laplacian

Graph Laplacian properties

- Suppose G is an undirected graph with non-negative weights
- Then 0 is an eigenvalue of L and the multiplicity k of the 0 eigenvalue is the number of connected components in the graph
- So if the clusters are disconnected, easy to tell how many clusters!
- If not, we still use the first k eigenvectors, but it's less obvious why... there are a few interpretations:
  - Graph cuts view
  - Random walks
  - Perturbation theory
  - (See the von Luxborg tutorial in the readings)

Solving normalized cut for 2 clusters

- Goal: Find a vector $\mathbf{x} \in \{-1, 1\}^N$, where $x_i = 1$ means data point i is in cluster A; otherwise cluster B
- Can show:

$$\begin{aligned} \min_{\mathbf{x}} Ncut(\mathbf{x}) \quad &= \quad \min_{\mathbf{y}} \frac{\mathbf{y}^T(D-W)\mathbf{y}}{\mathbf{y}^T D\mathbf{y}} \\ & \quad s.t. \quad \mathbf{y}^T D1 = 0, \mathbf{y} \text{ take discrete values} \end{aligned}$$

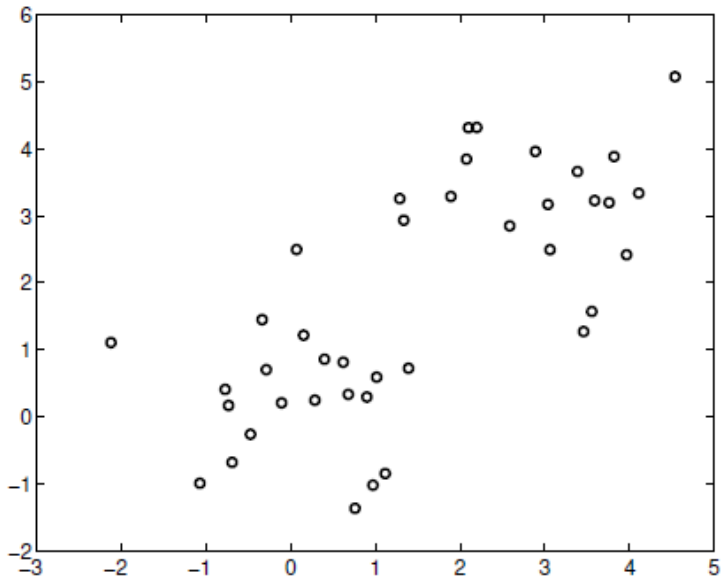
Solving normalized cut for 2 clusters

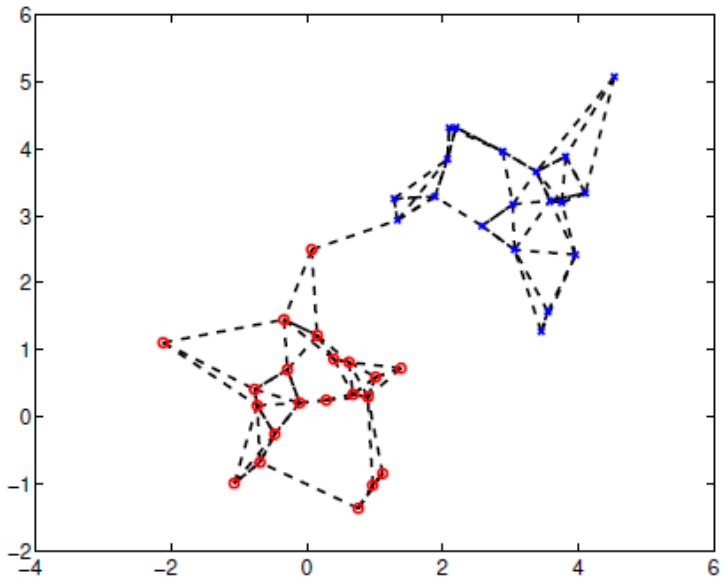
- The optimization problem is relaxed to solve for a continuous vector instead:

$$\begin{aligned} \min_{\mathbf{y}} \quad & \mathbf{y}^T(D-W)\mathbf{y} \\ s.t. \quad & \mathbf{y}^T D\mathbf{y} = 1 \\ \Longrightarrow \quad & (D-W)\mathbf{y} = \lambda D\mathbf{y} \end{aligned}$$

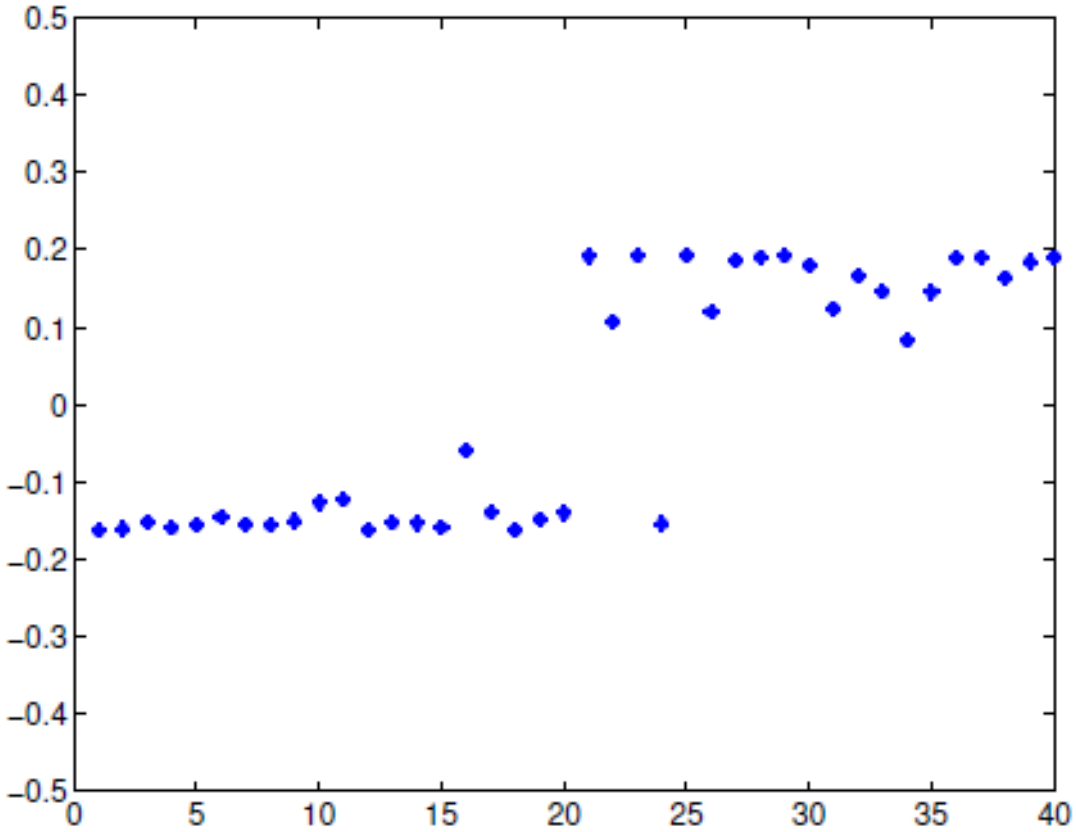
- The first eigenvector is all ones (all data in single cluster)
- We take the 2nd eigenvector as the real-valued solution, and threshold the result at 0 to discretize

Example





Example: The 2nd eigenvector

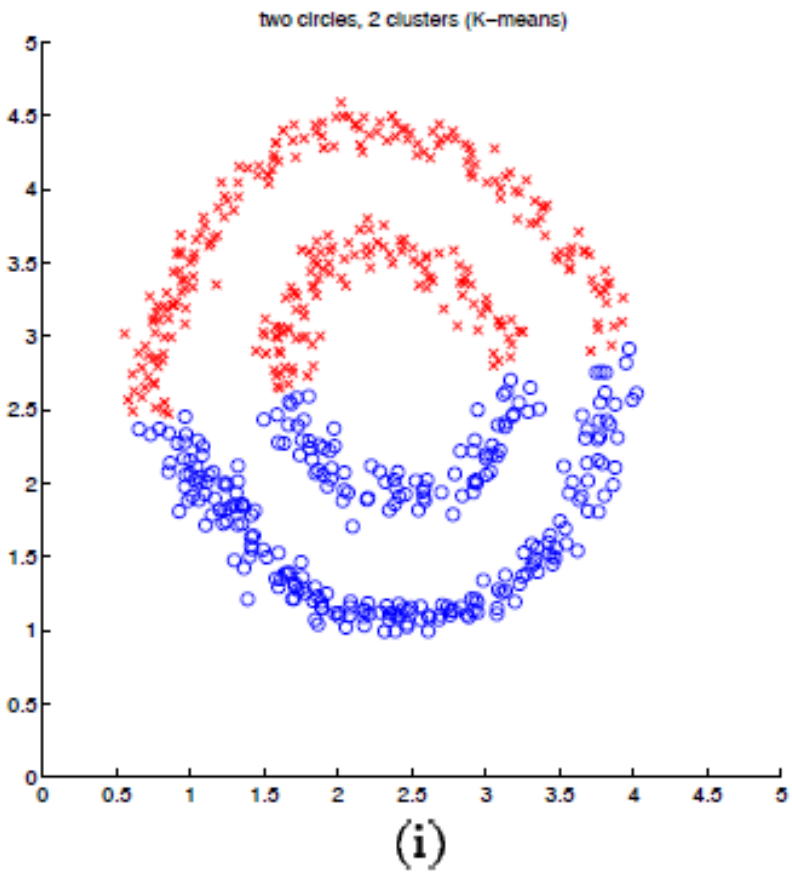


Beyond 2 clusters

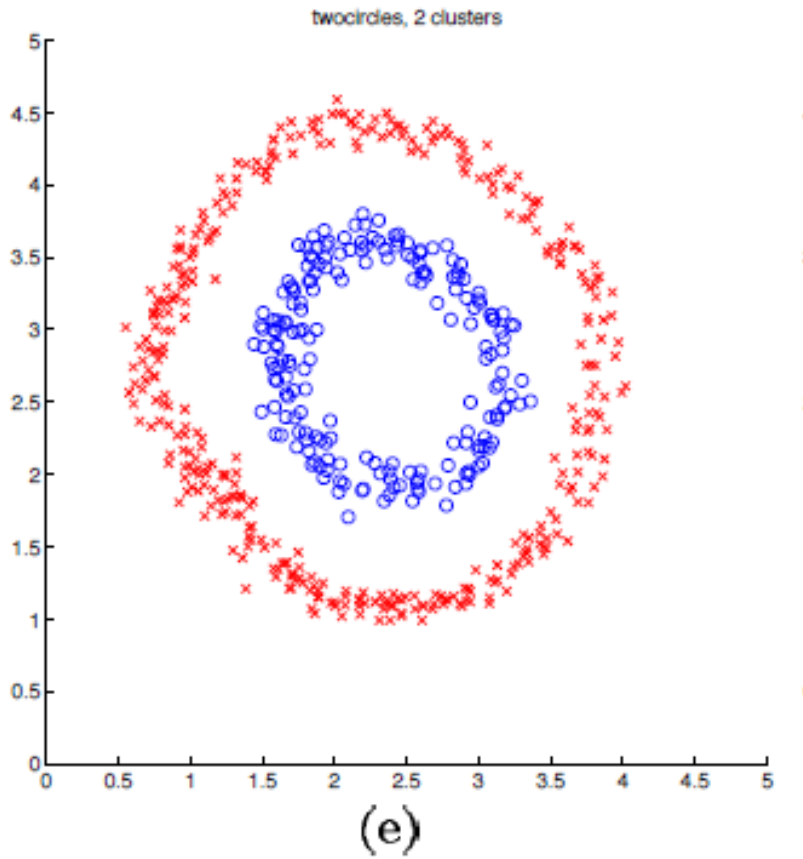
- Option 1: Recursively apply the 2-cluster algorithm
- Option 2: Treat the first k eigenvectors as a reduced-dimensionality representation of the data, and cluster the eigenvectors

Example: K-means vs. Spectral clustering

K-means

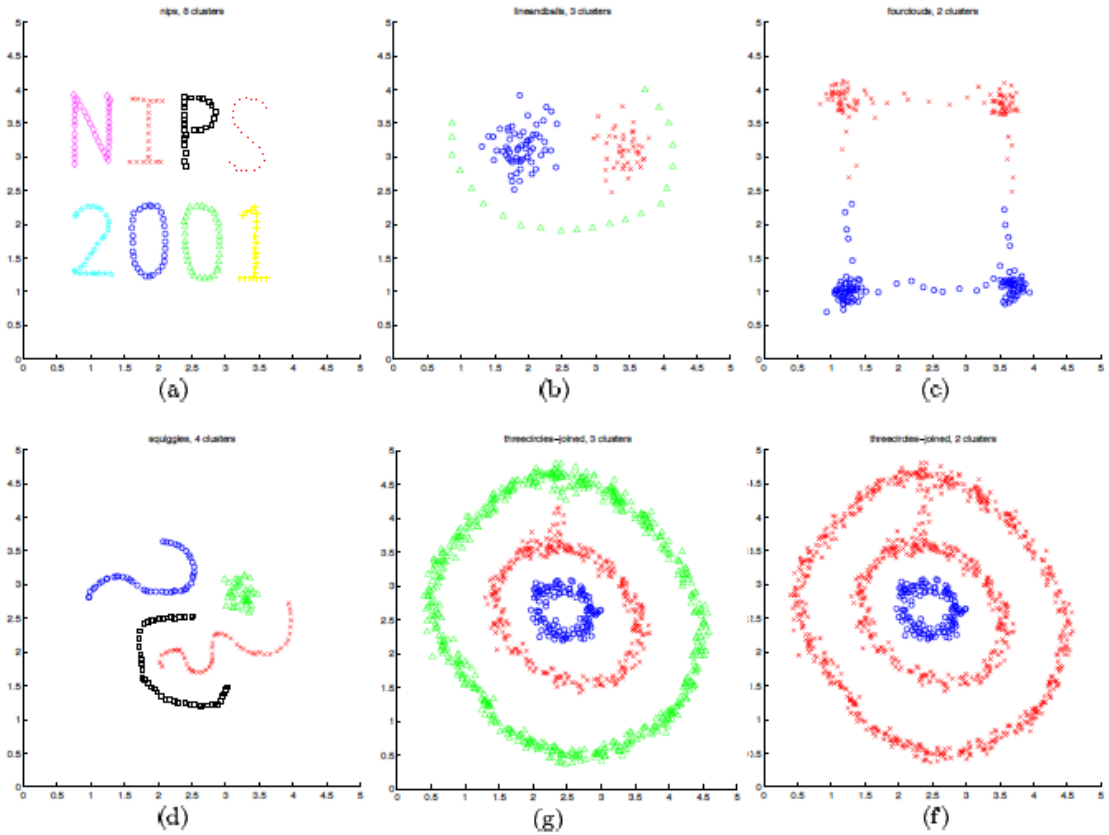


Spectral clustering



[Ng, Jordan, & Weiss]

Example: Spectral clustering

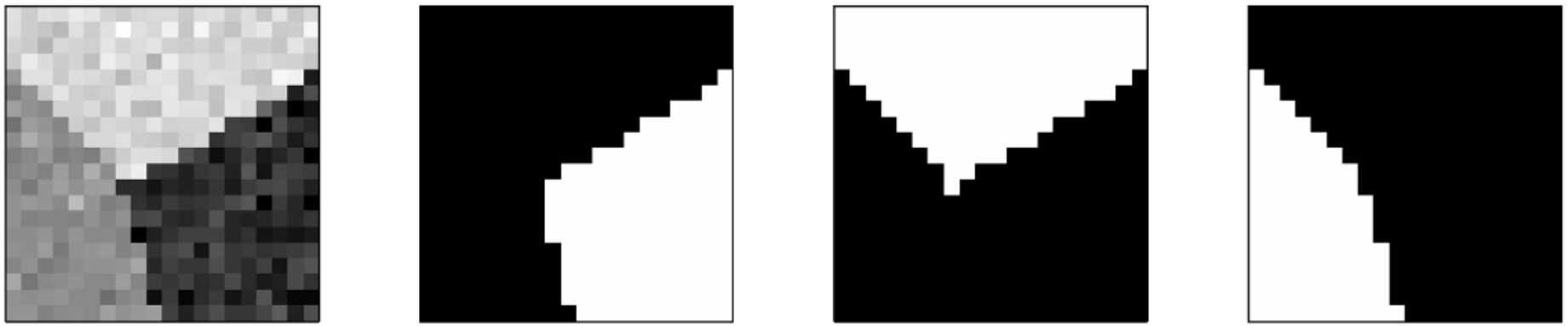


[Ng, Jordan, & Weiss]

Spectral clustering: real (but old) applications

Image segmentation: Each pixel i is a graph vertex, similarity depends on pixel content (color intensity) $F(i)$ and pixel location $X(i)$:

$$w_{ij} = e^{-\left(\frac{1}{2\pi\sigma^2}||F(i)-F(j)||^2+\frac{1}{2\pi\tau^2}||X(i)-X(j)||^2\right)}, \text{ if } ||X(i)-X(j)|| < r \text{ ; } 0 \text{ o.w.}$$



(a) (b) (c) (d)

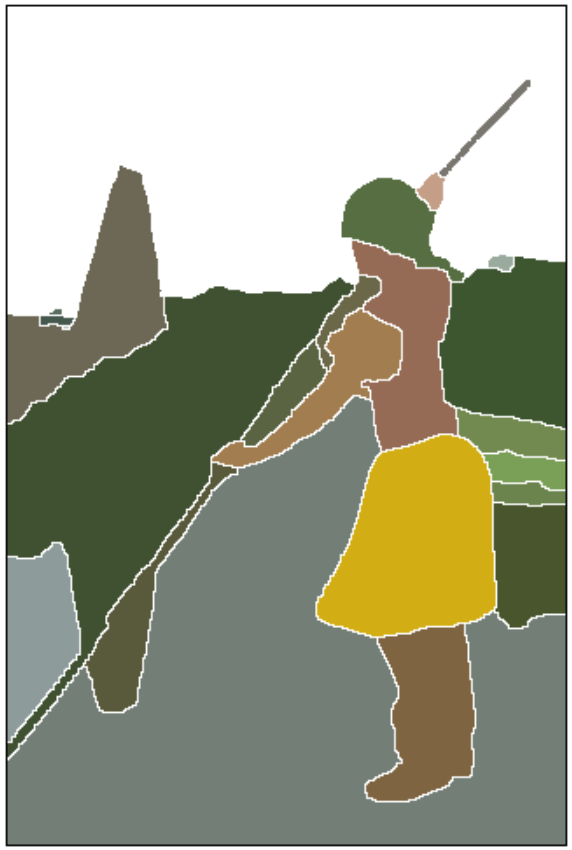
[Shi & Malik]

Spectral clustering: real (but old) applications

Image segmentation: Each pixel i is a graph vertex, similarity depends on pixel content (color intensity) $F(i)$ and pixel location $X(i)$:

$$w_{ij} = e^{-\left(\frac{1}{2\pi\sigma^2}||F(i)-F(j)||^2+\frac{1}{2\pi\tau^2}||X(i)-X(j)||^2\right)}, \text{ if } ||X(i)-X(j)|| < r \text{ ; } 0 \text{ o.w.}$$

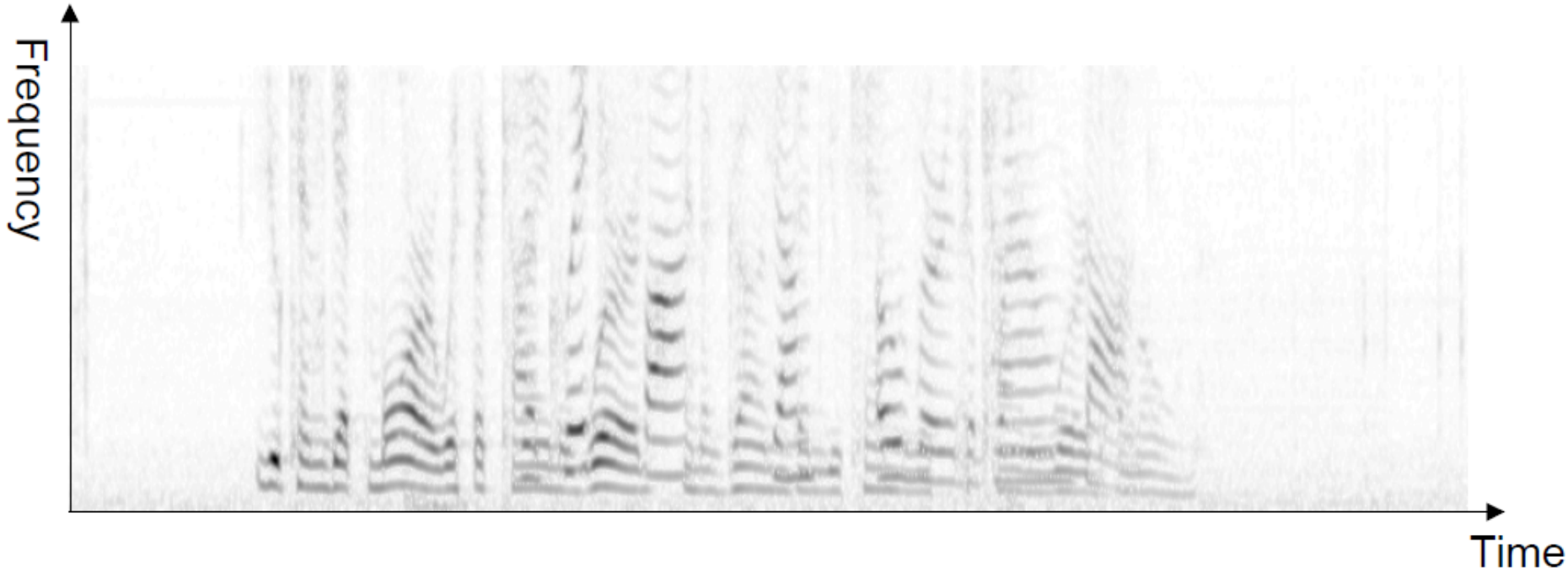




[Arbeláez *et al.*]

Spectral clustering: real applications

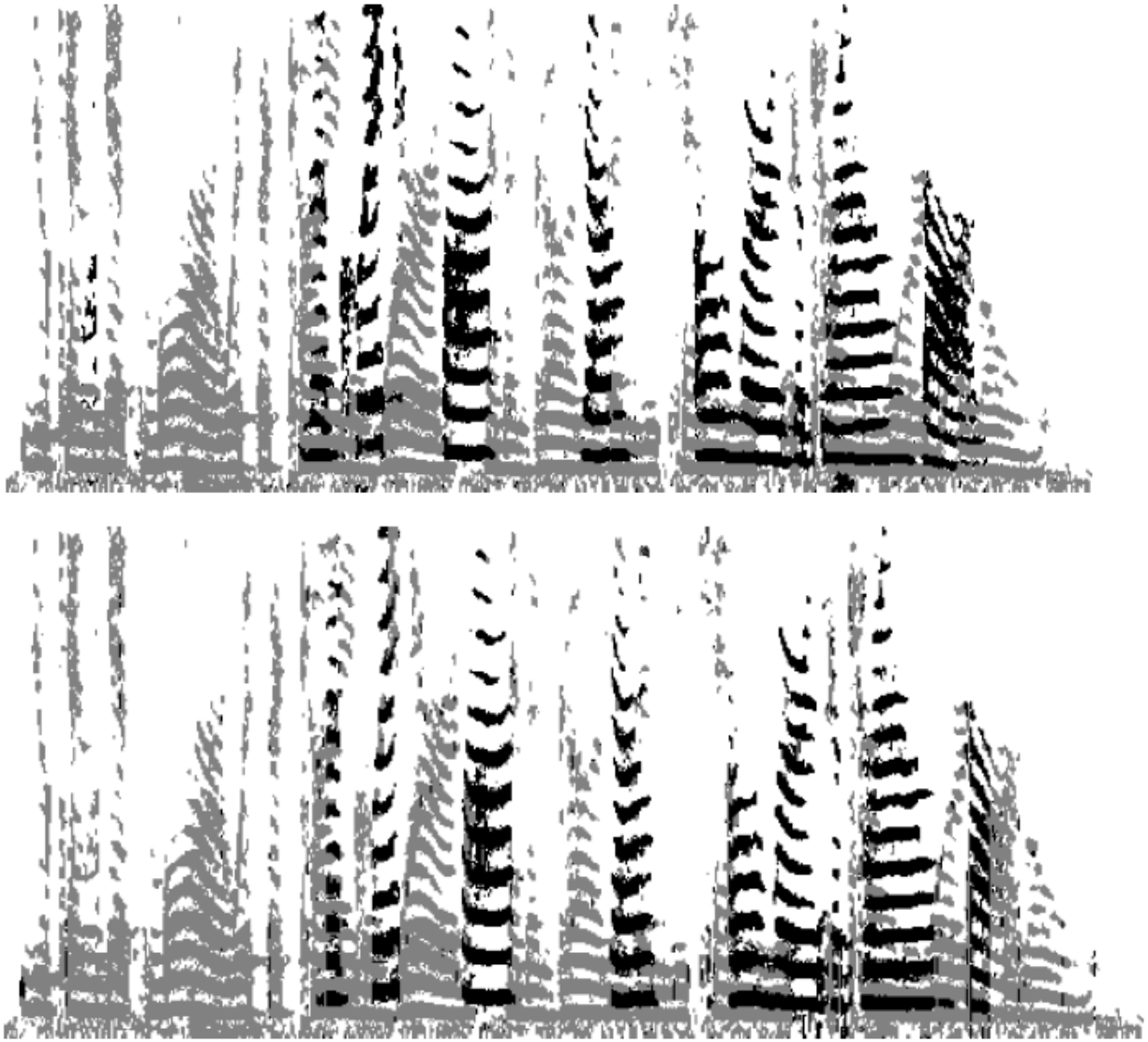
Speech separation (into speakers): Similar to image segmentation, where a "pixel" is a cell of a spectrogram:



[Bach & Jordan]

Spectral clustering: real applications

Speech segmentation (into speakers):



[Bach & Jordan]

Evaluating clustering

Suppose we have some labeled data. Let N be the number of labeled examples.

$$\text{purity}(\Omega, C) = \frac{1}{N} \sum_k \max_j |\omega_k \cap c_j|$$

where Ω is the proposed clustering, C is the ground-truth clustering, ω_k is the set of items assigned to cluster k, and c_j is the set of items with ground-truth label j.

- Ranges from 0 to 1
- Biased toward finer clusterings (purity $= 1$ if each example is in its own cluster)
- $\Rightarrow$ this measure makes sense for comparing clusterings with a given fixed number of clusters.

Evaluating clustering

Suppose we have some labeled data. Let N be the number of labeled examples. Then we define the normalized mutual information (NMI) as

$$\mathrm{NMI}(\Omega, C) = \frac{I(\Omega; C)}{[H(\Omega) + H(C)]/2}$$

Evaluating clustering

Suppose we have some labeled data. Let N be the number of labeled examples. Then we define the normalized mutual information (NMI) as

$$\mathrm{NMI}(\Omega, C) = \frac{I(\Omega; C)}{[H(\Omega) + H(C)]/2}$$

Define

- $p(k, j) = \frac{|\omega_k \cap c_j|}{N}$ as the probability that a randomly chosen example belongs to proposed cluster ω_k and ground-truth cluster c_j
- $p_\Omega(k) = \frac{|\omega_k|}{N}$ as the probability that a randomly chosen example belongs to proposed cluster ω_k, and similarly for $p_C(j)$

Evaluating clustering

Suppose we have some labeled data. Let N be the number of labeled examples. Then we define the normalized mutual information (NMI) as

$$\mathrm{NMI}(\Omega, C) = \frac{I(\Omega; C)}{[H(\Omega) + H(C)]/2}$$

Define

- $p(k, j) = \frac{|\omega_k \cap c_j|}{N}$ as the probability that a randomly chosen example belongs to proposed cluster ω_k and ground-truth cluster c_j
- $p_\Omega(k) = \frac{|\omega_k|}{N}$ as the probability that a randomly chosen example belongs to proposed cluster ω_k, and similarly for $p_C(j)$

Then

$$\mathrm{I}(\Omega, C) = \sum_k \sum_j p(k, j) \log \frac{p(k, j)}{p_\Omega(k) p_C(j)}$$

and the entropies $H(\cdot)$ are defined analogously

- Ranges from 0 and 1
- Mutual information alone would also favor over-clustering

From clustering to density estimation: Gaussian mixtures

From clustering to mixture models

Clustering makes a hard decision about cluster membership for each data point

- Can't always do out-of-sample extension
  - K-means: Can map new points to cluster with nearest cluster mean
  - No obvious way to do this for hierarchical clustering, spectral clustering

From clustering to mixture models

Clustering makes a hard decision about cluster membership for each data point

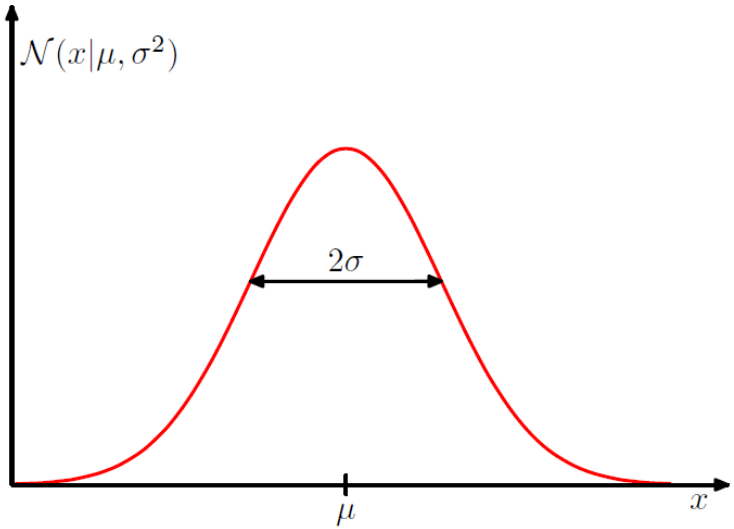
- Can't always do out-of-sample extension
  - K-means: Can map new points to cluster with nearest cluster mean
  - No obvious way to do this for hierarchical clustering, spectral clustering

Mixture models represent the density of the data as a mixture of component densities

- Can be seen as "soft" clustering
- Hard cluster ID can be assigned if needed
- New data points (soft-)assigned to clusters naturally
- We'll start with mixtures of Gaussians

Gaussian (normal) distributions

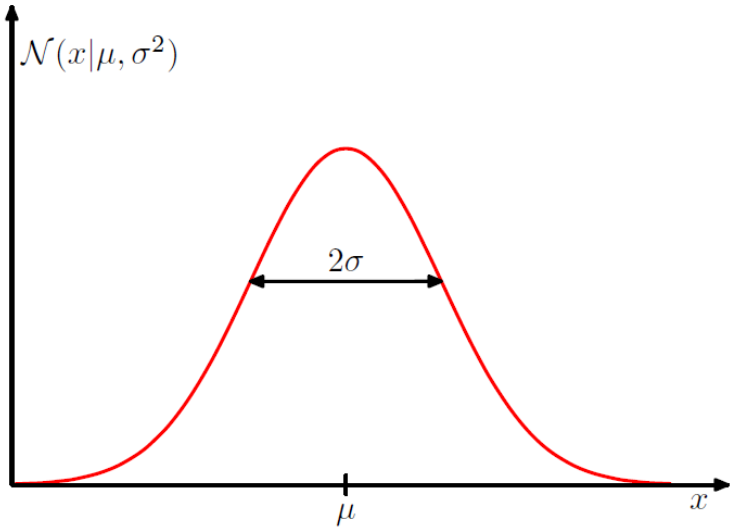
1-dimensional Gaussian density:



$$\mathcal{N}(x|\mu, \sigma^2) = \frac{1}{(2\pi\sigma^2)^{1/2}} \exp\left\{-\frac{1}{2\sigma^2(x-\mu)^2}\right\}$$

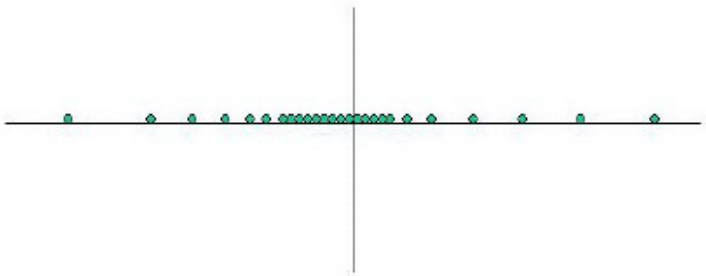
Gaussian (normal) distributions

1-dimensional Gaussian density:



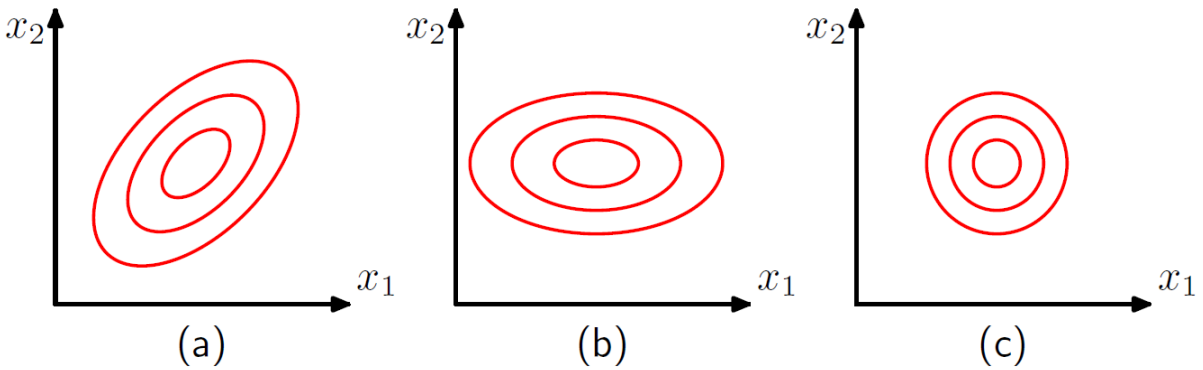
$$\mathcal{N}(x|\mu, \sigma^2) = \frac{1}{(2\pi\sigma^2)^{1/2}} \exp\left\{-\frac{1}{2\sigma^2(x-\mu)^2}\right\}$$

Random draws from a 1-D Gaussian:



Gaussian (normal) distributions

D-dimensional Gaussian density (here, $D = 2$):



$$\mathcal{N}(\mathbf{x}|\vec{\mu}, \vec{\Sigma}) = \frac{1}{(2\pi)^{D/2}} \frac{1}{|\Sigma|^{1/2}} \exp\left\{ -\frac{1}{2}(\mathbf{x} - \vec{\mu})^T \Sigma^{-1} (\mathbf{x} - \vec{\mu}) \right\}$$

Connection with Euclidean distance

Recall: log density of a Gaussian is
$\ln p(\vec{x}|\vec{\mu}, \vec{\Sigma}) = \text{const.} - \frac{1}{2}(\mathbf{x} - \vec{\mu})^T \vec{\Sigma^{-1}}(\mathbf{x} - \vec{\mu})$

- If $\Sigma = \vec{I}$ and dimensionality is D, then this equals
  $\text{const.} - \frac{1}{2}\sum_{d=1}^{D}(x_d - \mu_d)^2$
- $=$ squared Euclidean distance between $\mathbf{x}$ and $\vec{\mu}$ (scaled and shifted)

Maximum-likelihood (ML) estimation of Gaussian parameters $\vec{\mu}, \vec{\Sigma}$

Given data $\vec{X} = (\mathbf{x}_1, \ldots, \mathbf{x}_N)$, want to maximize data (log) prob

$$\ln p(\vec{X}|\vec{\mu}, \vec{\Sigma}) = -\frac{ND}{2}\ln(2\pi) - \frac{N}{2}\ln|\vec{\Sigma}| - \frac{1}{2}\sum_{n=1}^{N}(\mathbf{x}_n - \vec{\mu})^T\vec{\Sigma^{-1}}(\mathbf{x}_n - \vec{\mu})$$

Maximum-likelihood (ML) estimation of Gaussian parameters $\vec{\mu}, \vec{\Sigma}$

Given data $\vec{X} = (\mathbf{x}_1, \ldots, \mathbf{x}_N)$, want to maximize data (log) prob

$$\ln p(\vec{X}|\vec{\mu}, \vec{\Sigma}) = -\frac{ND}{2} \ln(2\pi) - \frac{N}{2} \ln |\vec{\Sigma}| - \frac{1}{2} \sum_{n=1}^{N} (\mathbf{x}_n - \vec{\mu})^T \vec{\Sigma^{-1}} (\mathbf{x}_n - \vec{\mu})$$

Setting the derivative to zero:

$$\frac{\partial}{\partial \vec{\mu}} \ln p(\vec{X}|\vec{\mu}, \vec{\Sigma}) = \sum_{n=1}^{N} \vec{\Sigma}^{-1} (\mathbf{x}_n - \vec{\mu}) = 0$$

Maximum-likelihood (ML) estimation of Gaussian parameters $\vec{\mu}, \vec{\Sigma}$

Given data $\vec{X} = (\mathbf{x}_1, \ldots, \mathbf{x}_N)$, want to maximize data (log) prob

$$\ln p(\vec{X}|\vec{\mu}, \vec{\Sigma}) = -\frac{ND}{2}\ln(2\pi) - \frac{N}{2}\ln|\vec{\Sigma}| - \frac{1}{2}\sum_{n=1}^{N}(\mathbf{x}_n - \vec{\mu})^T\vec{\Sigma^{-1}}(\mathbf{x}_n - \vec{\mu})$$

Setting the derivative to zero:

$$\frac{\partial}{\partial\vec{\mu}}\ln p(\vec{X}|\vec{\mu}, \vec{\Sigma}) = \sum_{n=1}^{N}\vec{\Sigma}^{-1}(\mathbf{x}_n - \vec{\mu}) = 0$$

which gives $\vec{\mu}_{ML} = \frac{1}{N}\sum_{n=1}^{N}\mathbf{x}_n$

Maximum-likelihood (ML) estimation of Gaussian parameters $\vec{\mu}, \vec{\Sigma}$

Given data $\vec{X} = (\mathbf{x}_1, \ldots, \mathbf{x}_N)$, want to maximize data (log) prob

$$\ln p(\vec{X}|\vec{\mu}, \vec{\Sigma}) = -\frac{ND}{2}\ln(2\pi) - \frac{N}{2}\ln|\vec{\Sigma}| - \frac{1}{2}\sum_{n=1}^{N}(\mathbf{x}_n - \vec{\mu})^T\vec{\Sigma^{-1}}(\mathbf{x}_n - \vec{\mu})$$

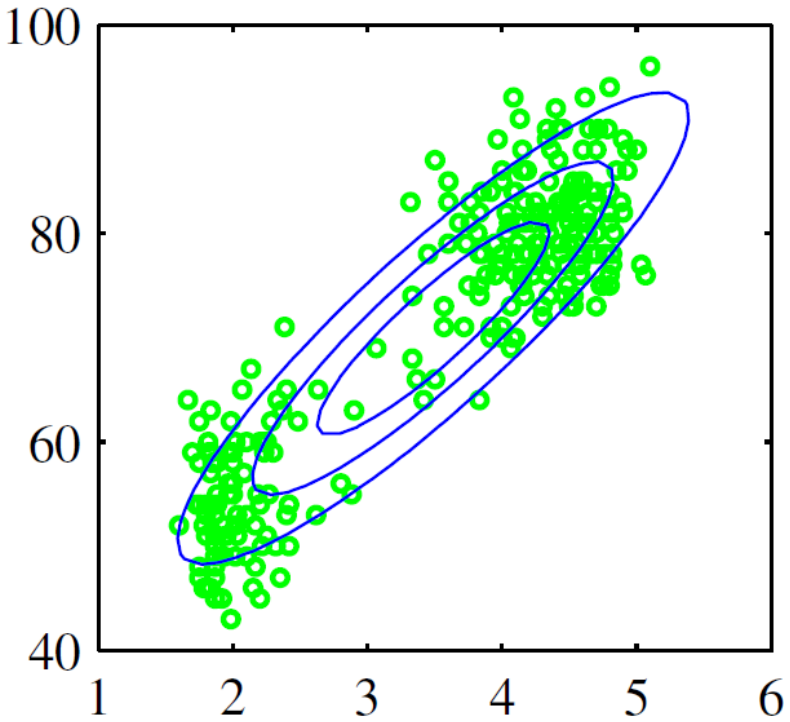
Setting the derivative to zero:

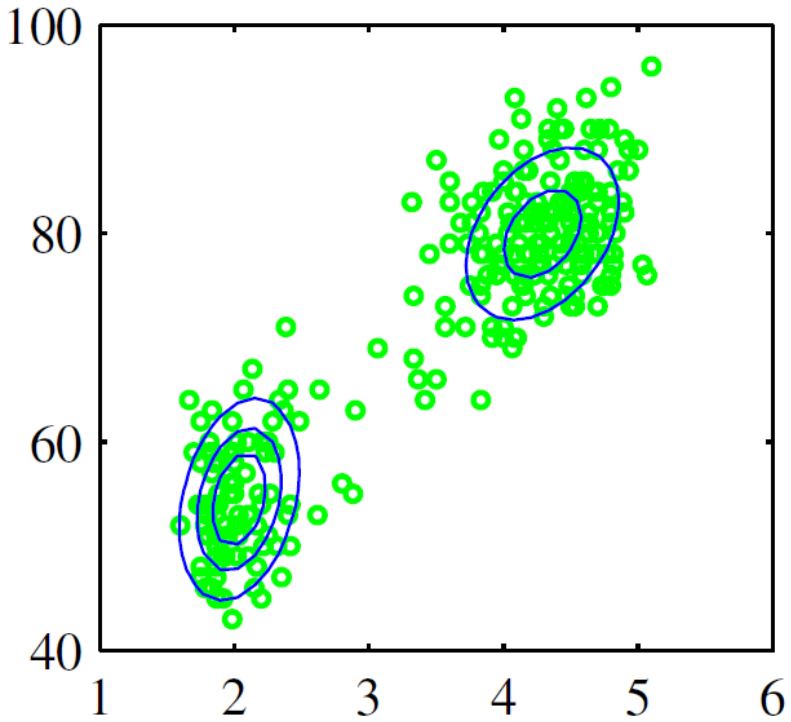
$$\frac{\partial}{\partial\vec{\mu}}\ln p(\vec{X}|\vec{\mu}, \vec{\Sigma}) = \sum_{n=1}^{N}\vec{\Sigma}^{-1}(\mathbf{x}_n - \vec{\mu}) = 0$$

which gives $\vec{\mu}_{ML} = \frac{1}{N}\sum_{n=1}^{N}\mathbf{x}_n$

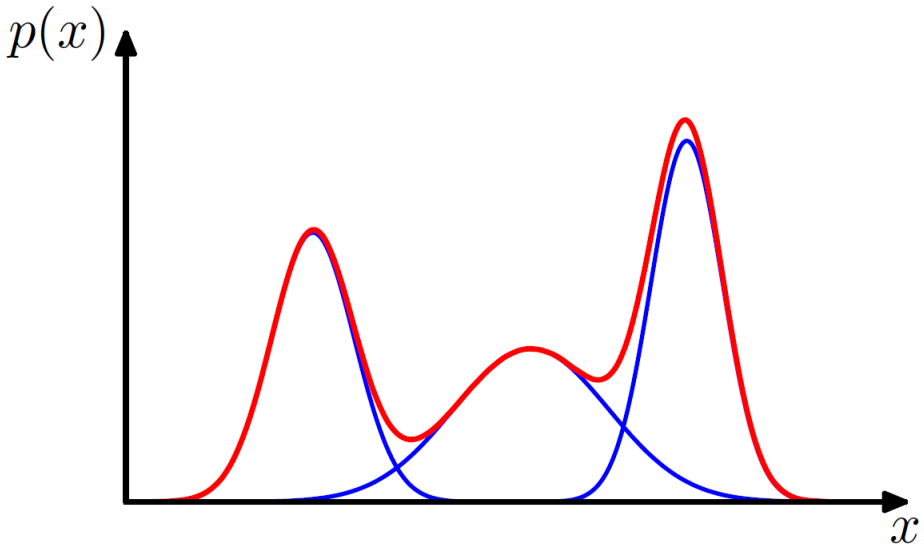
Similarly, $\vec{\Sigma}_{ML} = \frac{1}{N}\sum_{n=1}^{N}(\vec{x}_n - \vec{\mu}_{ML})(\vec{x}_n - \vec{\mu}_{ML})^T$

Gaussian mixtures (Gaussian mixture models, GMM)





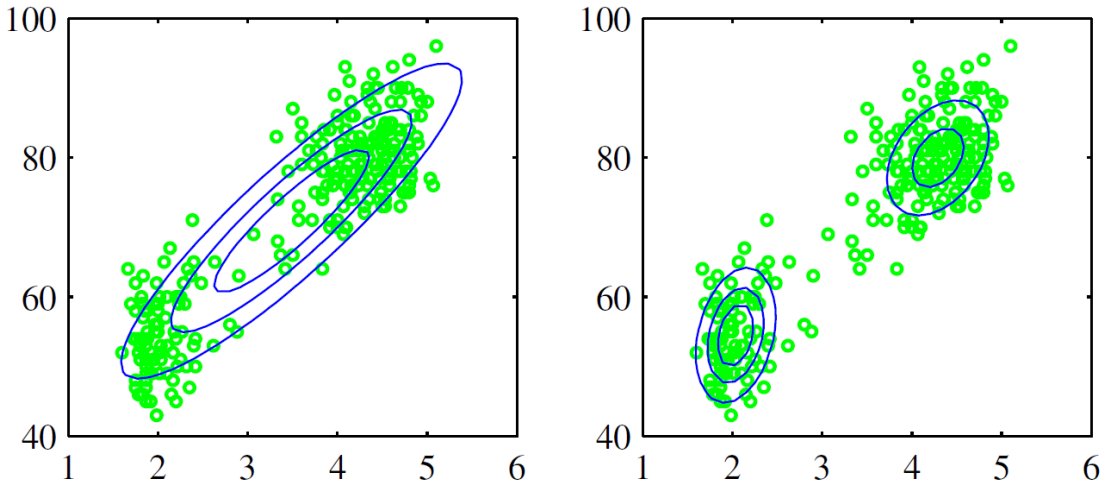
Gaussian mixtures



$$p(\mathbf{x}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}|\vec{\mu}_k, \vec{\Sigma}_k)$$

$$\pi_k \geq 0, \ \sum_{k=1}^{K} \pi_k = 1$$

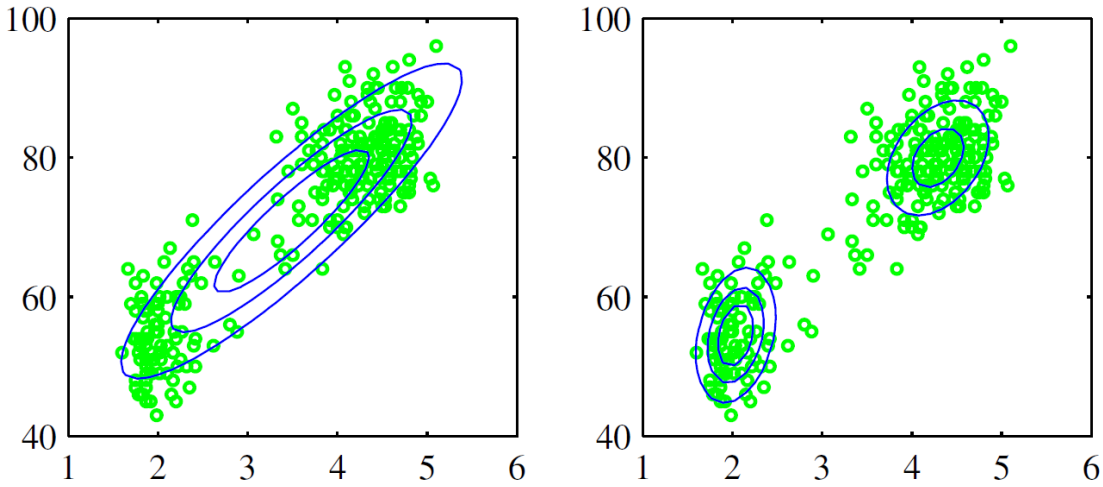
Gaussian mixtures



Motivations:

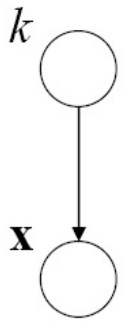
- GMs can approximate essentially arbitrary distributions
- Mixtures express the existence of *latent (hidden) variables*

Gaussian mixtures



Motivations:

- GMs can approximate essentially arbitrary distributions
- Mixtures express the existence of *latent (hidden) variables*
- Graphical model notation *a la* Bishop textbook:



$$p(\mathbf{x}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}|\vec{\mu}_k, \vec{\Sigma}_k)$$

Gaussian mixtures: Issues with ML estimation

$$\ln p(X|\pi,\theta) = \sum_{i=1}^{N} \ln \sum_{k=1}^{K} \pi_k N(x_i|\mu_k, \Sigma_k)$$

- No closed-form solution
- Infinite solution: Just set $\mu_1 = x_1$, let $\sigma_1 \to 0$
- How many Gaussians?

Gaussian mixtures: Issues with ML estimation

$$\ln p(X|\pi,\theta) = \sum_{i=1}^{N} \ln \sum_{k=1}^{K} \pi_k N(x_i|\mu_k, \Sigma_k)$$

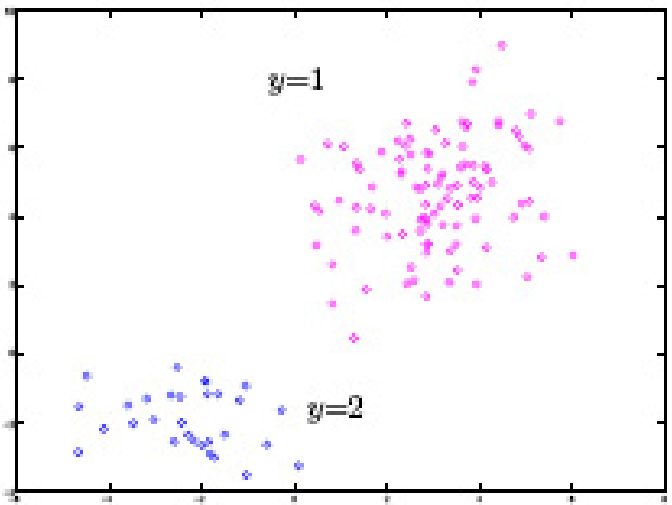
- No closed-form solution
- Infinite solution: Just set $\mu_1 = x_1$, let $\sigma_1 \to 0$
- How many Gaussians?
- (How about one for each data point?)
- Need *regularization* – we'll get back to that later

Gaussian mixtures: ML estimation

- Introduce a set of binary indicator variables $z_{i1}, \ldots, z_{iK}$, where $z_{ik} = 1$ if x_i came from Gaussian component k and $z_{ik} = 0$ otherwise
- Count of examples from k^{th} component: $N_k = \sum_{i=1}^{N} z_{ik}$

ML estimation with known component labels

- If we know z_i, then the ML estimates of the Gaussian components are just like in the single-Gaussian case:



$$\hat{\pi}_k = \frac{N_k}{N}$$

$$\hat{\mu}_k = \frac{1}{N_k} \sum_{i=1}^{N} z_{ik} x_i$$

$$\hat{\Sigma}_k = \frac{1}{N\,k} \sum_{i=1}^{N} z_{ik} (x_i - \hat{\mu}_k)(x_i - \hat{\mu}_k)^T$$

ML estimation with unknown component labels: The credit assignment problem

- When we don't know $\mathbf{z}_i$, we face a credit assignment problem: Which component was reponsible for $\mathbf{x}_i$?

ML estimation with unknown component labels: The credit assignment problem

- When we don't know $\mathbf{z}_i$, we face a credit assignment problem: Which component was reponsible for $\mathbf{x}_i$?
- Suppose we *do* know the component parameters $\vec{\theta} = \{\vec{\mu}_k, \vec{\Sigma}_k\}$
- Then the *posterior probability* of the indicator variables is (using Bayes' rule)

$$\gamma_{ik} = P(z_{ik} = 1 | \mathbf{x}_i, \vec{\theta}) = \frac{\pi_k p(\mathbf{x}_i | \vec{\mu}_k, \vec{\Sigma}_k)}{\sum_{l=1}^{K} \pi_l p(\mathbf{x}_i | \vec{\mu}_l, \vec{\Sigma}_l)}$$

- γ_{ik} is called the *responsibility* of the kth component for $\mathbf{x}_i$. Note that $\sum_{k=1}^{K} \gamma_{ik} = 1$

ML estimation with unknown component labels (cont'd)

Now, "pretend" that the γ_{ik} are the indicator variables (instead of z_{ik}), i.e., that $N_k = \sum_{i=1}^N \gamma_{ik}$:

$$\begin{aligned}
\hat{\pi}_k &= \frac{\sum_{i=1}^N \gamma_{ik}}{N} \\
\hat{\vec{\mu}}_k &= \frac{1}{\sum_{i=1}^N \gamma_{ik}} \sum_{i=1}^N \gamma_{ik} \mathbf{x}_i \\
\hat{\vec{\Sigma}}_k &= \frac{1}{\sum_{i=1}^N \gamma_{ik}} \sum_{i=1}^N \gamma_{ik} (\mathbf{x}_i - \hat{\vec{\mu}}_k)(\mathbf{x}_i - \hat{\vec{\mu}}_k)^T
\end{aligned}$$

Summary so far

- If we know the *parameters* and *indicators* (component labels) then we are done

Summary so far

- If we know the *parameters* and *indicators* (component labels) then we are done
- If we know the *indicators* but not the parameters, we can do ML estimation of the parameters – and we are done

Summary so far

- If we know the *parameters* and *indicators* (component labels) then we are done
- If we know the *indicators* but not the parameters, we can do ML estimation of the parameters – and we are done
- If we have a *guess for the parameters*, we can compute the posteriors of indicators; then estimate parameters that maximize the expected likelihood – and then we are done

Summary so far

- If we know the *parameters* and *indicators* (component labels) then we are done
- If we know the *indicators* but not the parameters, we can do ML estimation of the parameters – and we are done
- If we have a *guess for the parameters*, we can compute the posteriors of indicators; then estimate parameters that maximize the expected likelihood – and then we are done
- In reality, we know neither the parameters nor the indicators

The expectation-maximization (EM) algorithm

- Initialization: Guess $\vec{\theta}, \vec{\pi}$
- Iterate:

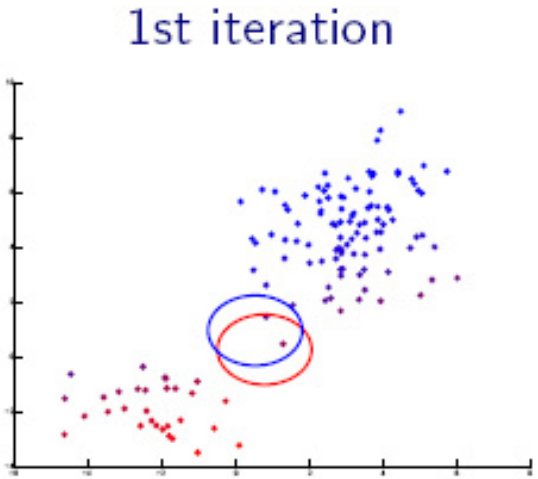
  E-step: Compute γ_{ik} using current estimates of $\vec{\theta}, \vec{\pi}$

  M-step: Estimate new parameters, maximizing the expected likelihood, given the current γ_{ik}

- Until log likelihood converges

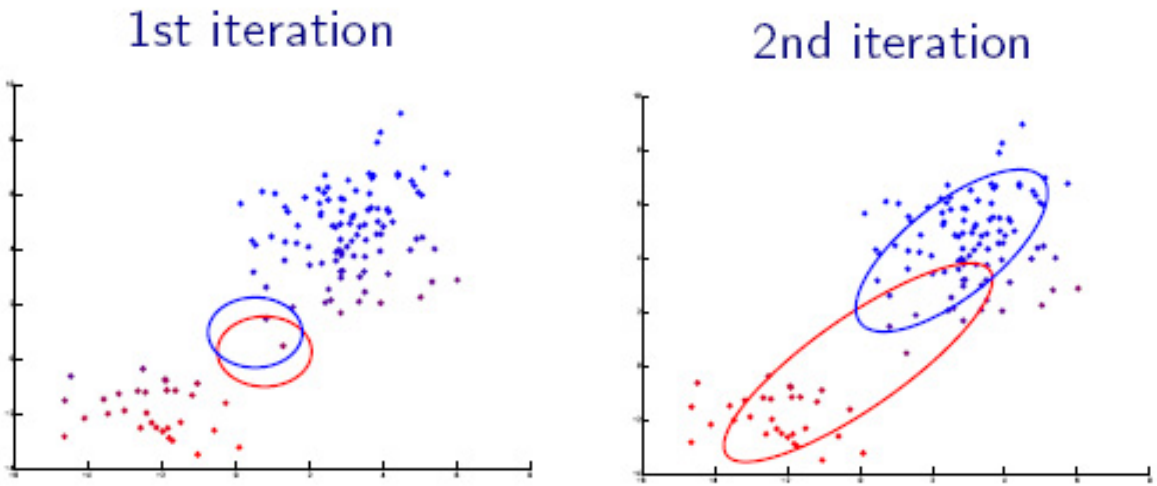
EM for Gaussian mixtures: Example

Colors represent γ_{ik} after the E-step



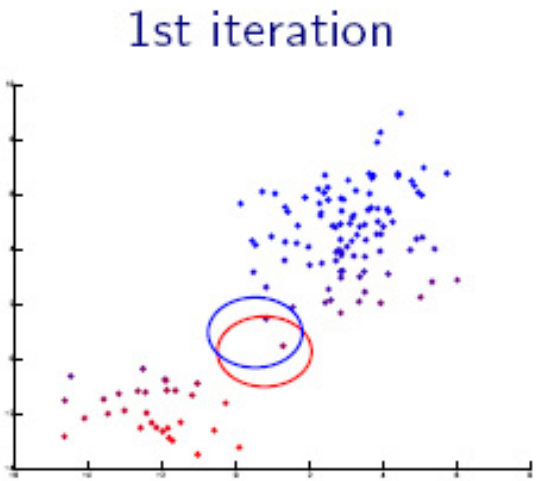
EM for Gaussian mixtures: Example

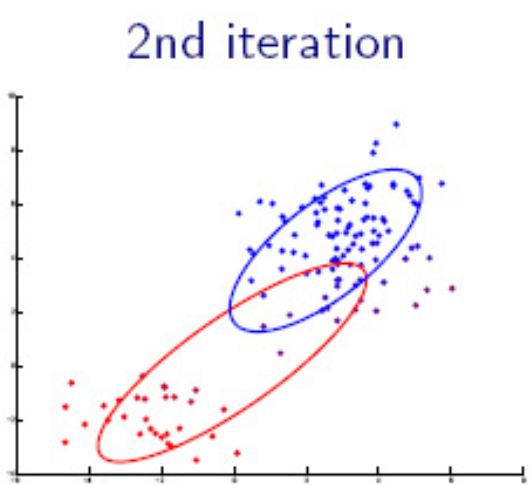
Colors represent γ_{ik} after the E-step

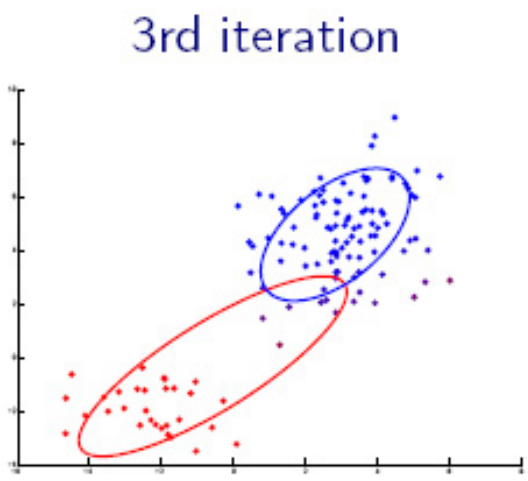


EM for Gaussian mixtures: Example

Colors represent γ_{ik} after the E-step

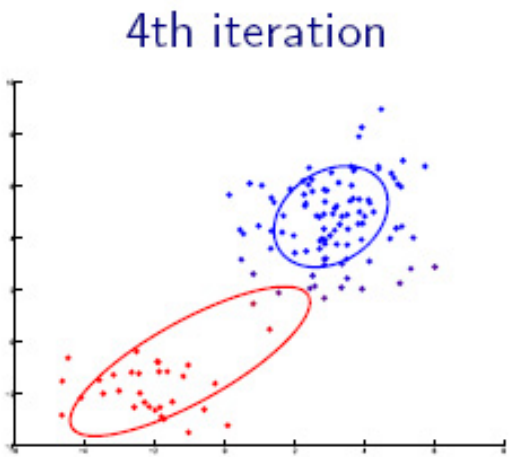






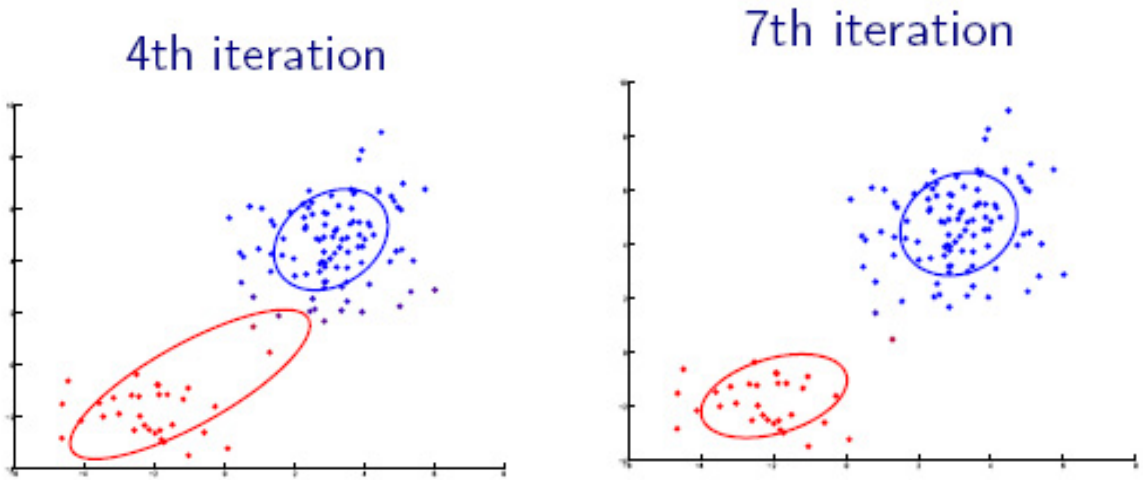
EM for Gaussian mixtures: Example

Colors represent γ_{ik} after the E-step



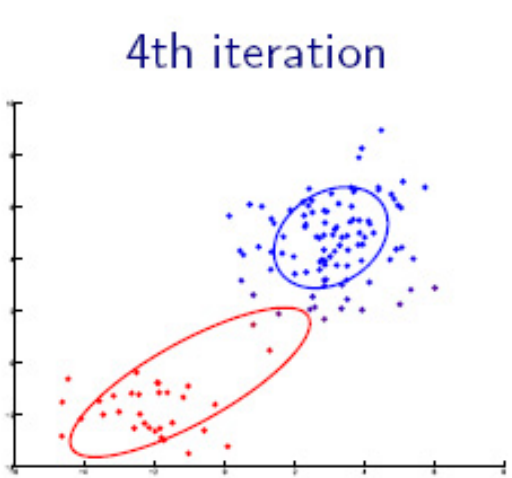
EM for Gaussian mixtures: Example

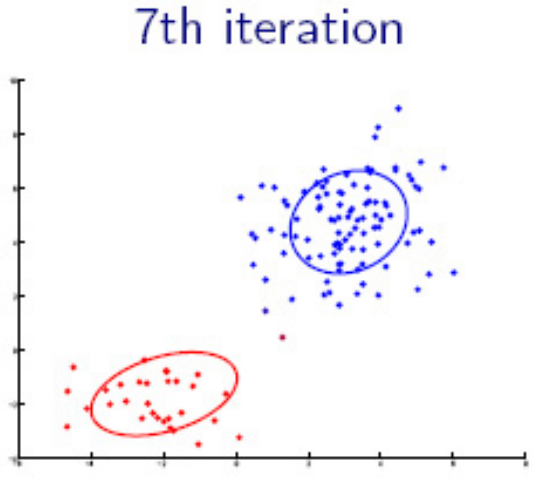
Colors represent γ_{ik} after the E-step

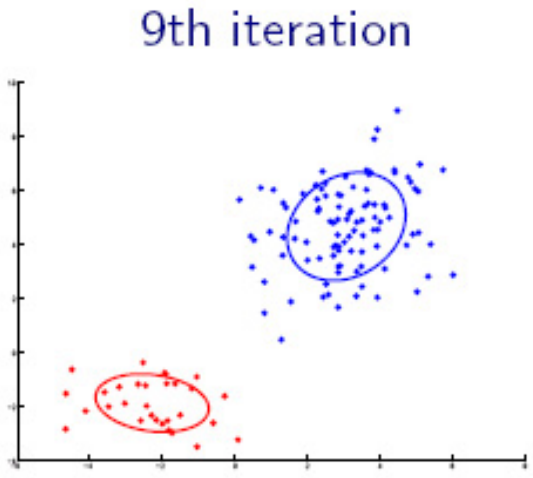


EM for Gaussian mixtures: Example

Colors represent γ_{ik} after the E-step







Convergence of EM

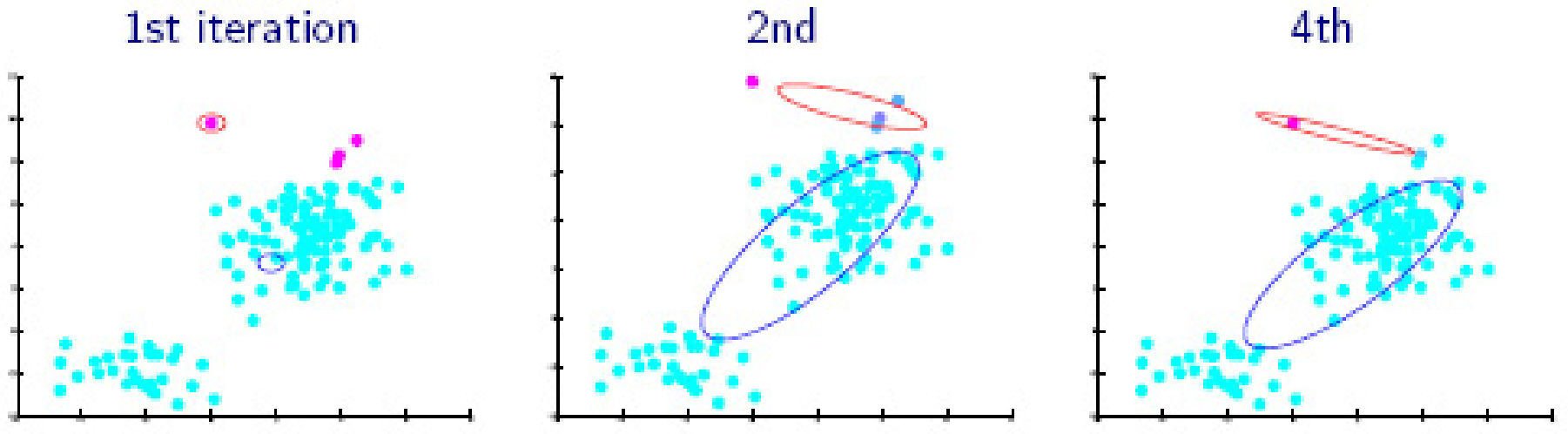
- Let $\ell^{(t)}$ be $\ln p(\vec{X}|\hat{\vec{\mu}}, \hat{\vec{\Sigma}}, \hat{\vec{\pi}})$ after t iterations
- Can show:

  $$\ell^{(0)} \leq \ell^{(1)} \leq \ldots \leq \ell^{(t)} \ldots$$

- EM converges, but possibly to a local maximum of the likelihood
- One solution (as in k-means): Use multiple initializations and pick the result with highest likelihood

Issues with EM for GMs

We can be very unlucky with the initialization:



Why? $\lim_{\sigma^2 \to 0} \mathcal{N}(\mathbf{x}|\vec{\mu} = \mathbf{x}, \vec{\Sigma} = \sigma^2 \vec{I}) = \infty$

Regularizing EM

- Impose a prior distribution on $\vec{\theta}$
- Instead of maximizing the expected likelihood in the M-step, maximize the *posterior* probability of $\vec{\theta}$

$$\vec{\theta} = \operatorname*{argmax}_{\theta} E_{z_{ik}|\vec{X},\vec{\pi},\vec{\theta}}[\ln p(\vec{X}, \vec{Z}|\vec{\pi}, \vec{\theta})] + \ln p(\vec{\theta})$$

- This is the *maximum a posteriori* (MAP) estimate (as opposed to ML)
- In practice: Often just impose a minimum variance on each dimension of each component

Issues with EM for GMs: What is K?

- This is the *model selection* problem
- Idea 1: Choose K to maximize the likelihood

Issues with EM for GMs: What is K?

- This is the *model selection* problem
- Idea 1: Choose K to maximize the likelihood
  - Result: A separate, tiny Gaussian for each training example
  - In the limit $\vec{\Sigma} \rightarrow 0$, this yields infinite likelihood

Issues with EM for GMs: What is K?

- This is the *model selection* problem
- Idea 1: Choose K to maximize the likelihood
  - Result: A separate, tiny Gaussian for each training example
  - In the limit $\vec{\Sigma} \to 0$, this yields infinite likelihood
- Some solutions involve optimizing the likelihood plus a "complexity" penalty

Regularization & model selection for GMs, in practice

In practice, most often we use some held-out data to tune *hyperparameters* like K or the minimum variance

Regularization & model selection for GMs, in practice

In practice, most often we use some held-out data to tune *hyperparameters* like K or the minimum variance

- Divide training set into *train* and *development* (or *held-out*, or *validation*) sets
- For each choice of hyperparameters, estimate parameters via EM on the training set and test on the development set
- Choose those hyperparameters that yield the best dev set performance

Regularization & model selection for GMs, in practice

In practice, most often we use some held-out data to tune *hyperparameters* like K or the minimum variance

- Divide training set into *train* and *development* (or *held-out*, or *validation*) sets
- For each choice of hyperparameters, estimate parameters via EM on the training set and test on the development set
- Choose those hyperparameters that yield the best dev set performance
- Another way of choosing K:
  - Start with a single Gaussian, do EM until convergence
  - Repeat: *Split* each Gaussian into two by adding small random values to parameters, run EM, test on dev set
  - Until no performance improvement on dev set

Aside: Connection between EM and gradient descent for Gaussian mixtures

Xu and Jordan, "On Convergence Properties of the EM Algorithm for Gaussian Mixtures," *Neural Computation* **8**(1)129-151, 1996.

- EM update can be obtained from the gradient via a projection matrix
- Convergence rates of EM vs. gradient methods can depend on shape of the likelihood function
- Advantages of EM: No learning rate to tune, no need to add constraints

Latent variable models beyond GMs: Graphical models

- Graphical language for representing and reasoning about random variables and their distributions
- Probability distributions (for a *single* random variable) $p(X)$
  discrete $X \Rightarrow$ PMF
  continuous $X \Rightarrow$ density

Statistical independence

- Definition and implications of independence:

$$\boxed{X \perp Y} \quad \Leftrightarrow \quad p(x \mid y) = p(x)$$

$$\Updownarrow \qquad\qquad \Updownarrow$$

$$p(x, y) = p(x)p(y) \quad \Leftrightarrow \quad p(y \mid x) = p(y)$$

Statistical independence

- Definition and implications of independence:

$$\boxed{X \perp Y} \iff p(x \mid y) = p(x)$$

$$\Updownarrow \qquad\qquad \Updownarrow$$

$$p(x, y) = p(x)\,p(y) \iff p(y \mid x) = p(y)$$

- Conditional independence:

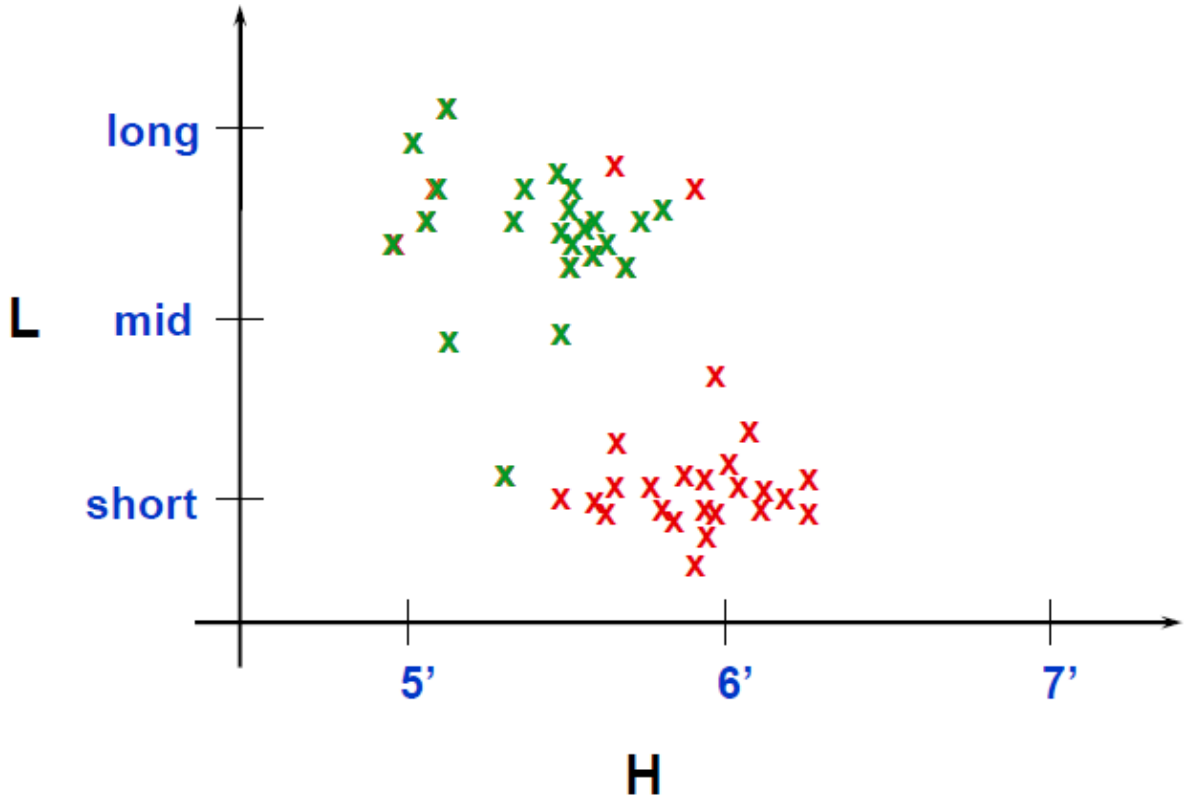
$$\boxed{X \perp Y \mid Z} \iff p(x \mid y, z) = p(x \mid z)$$

$$\Updownarrow \qquad\qquad \Updownarrow$$

$$p(x, y \mid z) = p(x \mid z)\,p(y \mid z) \iff p(y \mid x, z) = p(y \mid z)$$

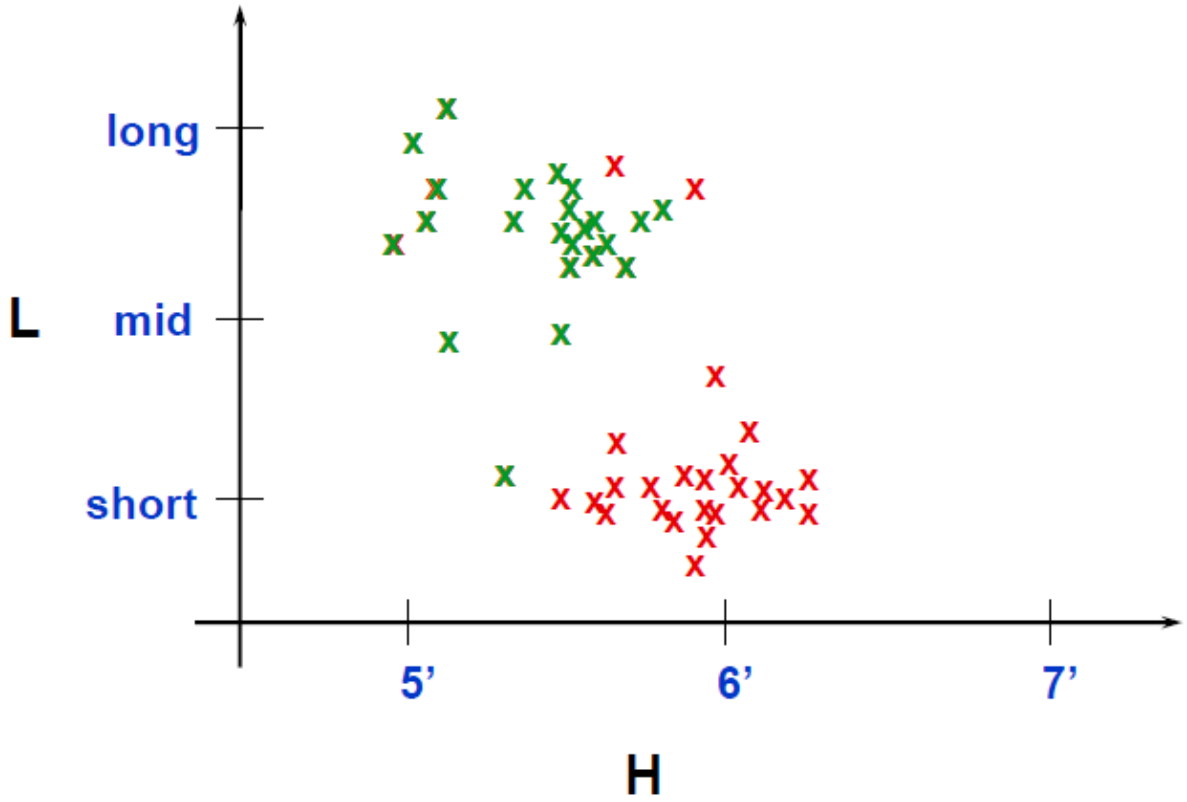
Is hair length independent of height?

- Measurements from a group of people:



Is hair length independent of height?

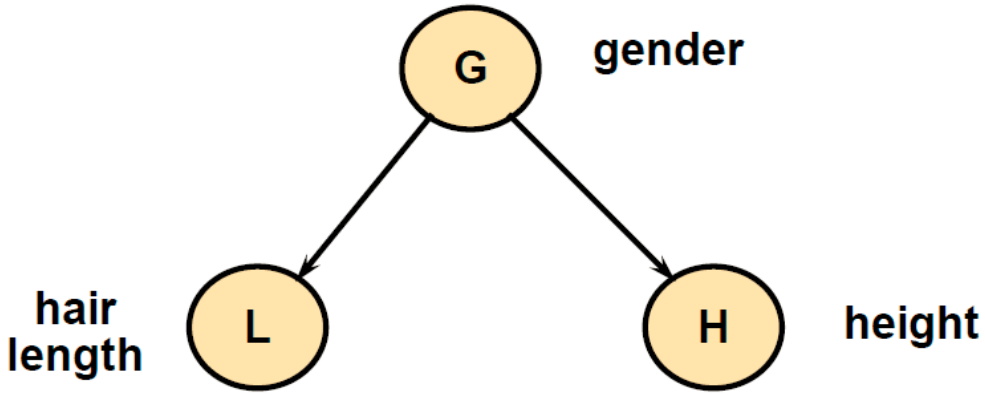
- Measurements from a group of people:



- Not in general; yes if we know the gender

Hair length and height

- We can draw it as a graph:

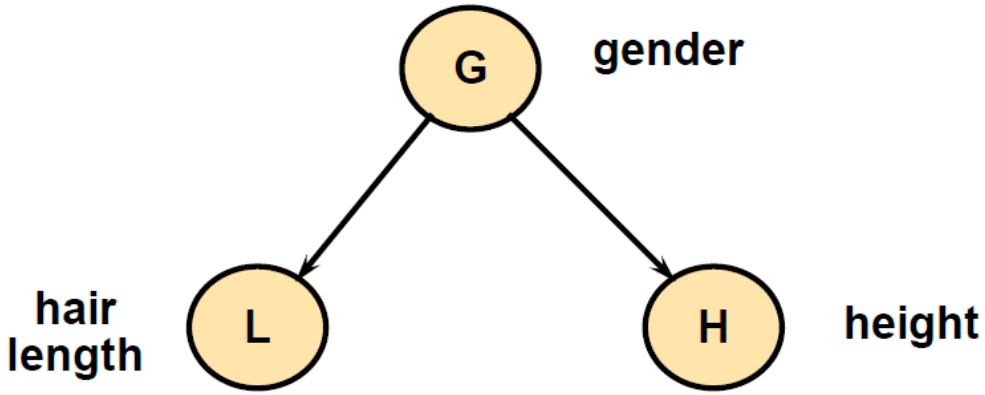


$$p(h \mid l) \neq p(h) \qquad H \not\perp L$$
$$p(h \mid l, g) = p(h \mid g) \qquad H \perp L \mid G$$

- This is the *common cause* situation: the latent variable G affects both H and L;
  H "affects" L only through G

Hair length and height

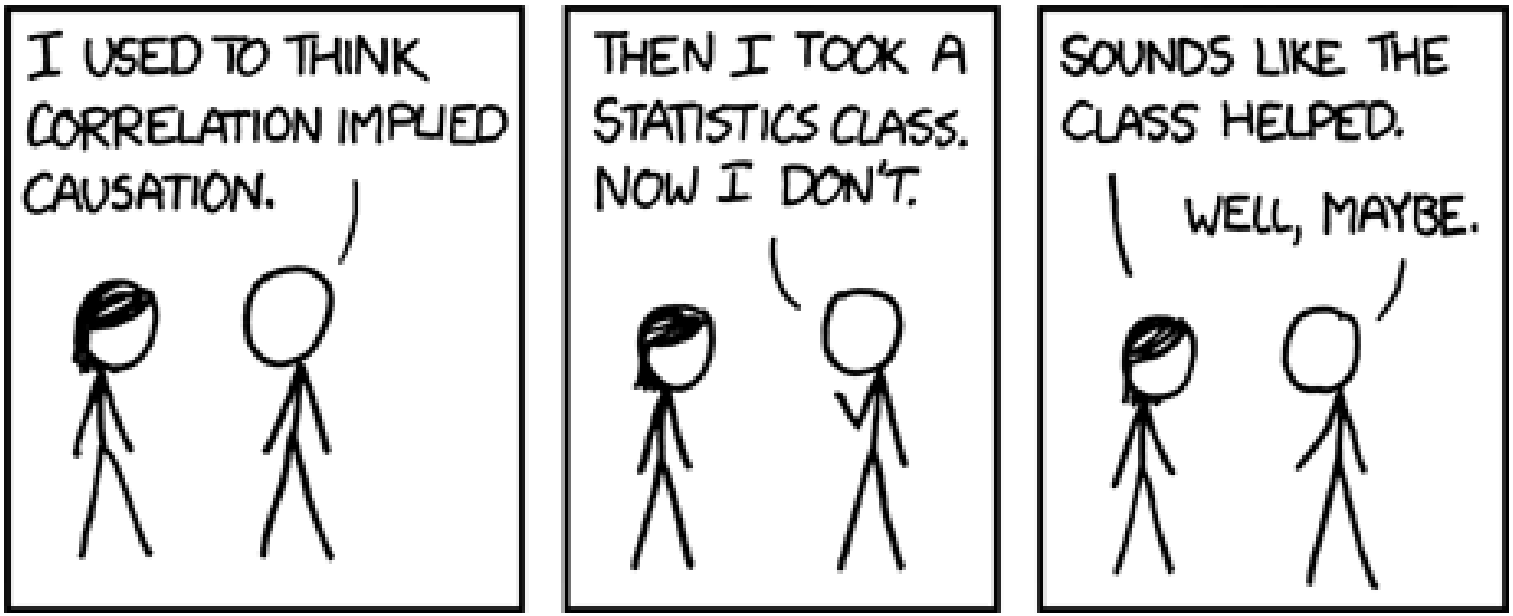
- We can draw it as a graph:



$$p(h \mid l) \neq p(h) \qquad H \not\perp L$$
$$p(h \mid l, g) = p(h \mid g) \qquad H \perp L \mid G$$

- This is the *common cause* situation: the latent variable G affects both H and L;
  H "affects" L only through G
- Note: "affects" $\neq$ "causes"

Correlation vs causation

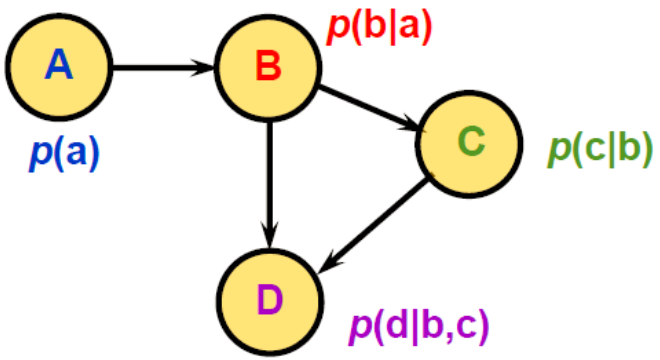


source: XKCD

Bayesian networks

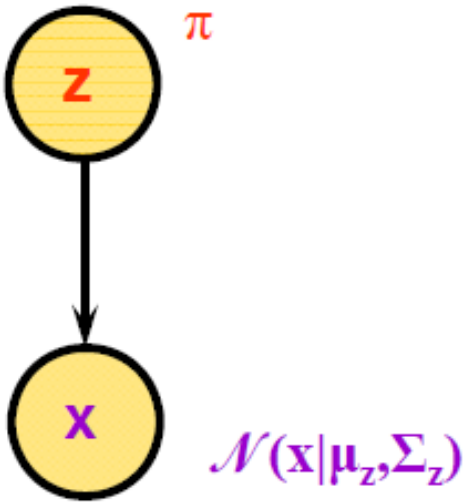
- Directed graphical model: a graph over random variables $X_1, \ldots, X_n$
- Must be a directed acyclic graph (DAG)
- Node (vertex) X_i with parents $\pi(X_i)$: local probability function $p\left(X_i | \pi(X_i)\right)$
- Joint probability:

$$p(X_1, \ldots, X_n) = \prod_{i=1}^{n} p\left(X_i | \pi(X_i)\right)$$

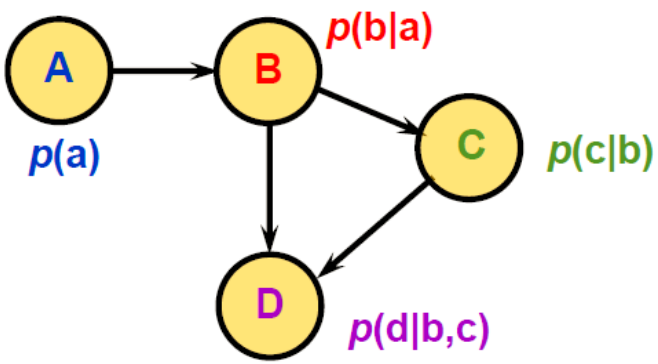


$\Rightarrow$ $p(a,b,c,d) = p(a)\, p(b|a)\, p(c|b)\, p(d|b,c)$

Bayesian network for Gaussian mixture



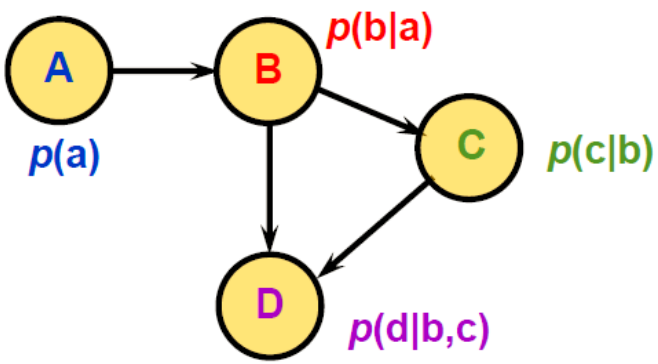
Factorization in BN



$\Rightarrow$ $p(a,b,c,d) = p(a)\,p(b|a)\,p(c|b)\,p(d|b,c)$

- Suppose a, b, c, d are discrete and each have 10 values
- Representing $p(a)$ requires 10 numbers (9, really)
- Representing $p(a|b)$ requires

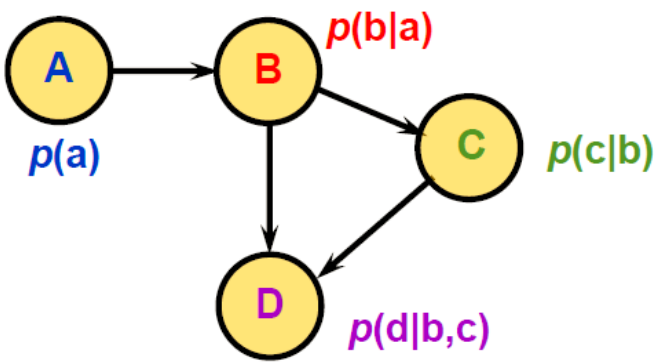
Factorization in BN



$\Rightarrow$ $p(a,b,c,d) = p(a)\,p(b|a)\,p(c|b)\,p(d|b,c)$

- Suppose a, b, c, d are discrete and each have 10 values
- Representing $p(a)$ requires 10 numbers (9, really)
- Representing $p(a|b)$ requires 100 numbers

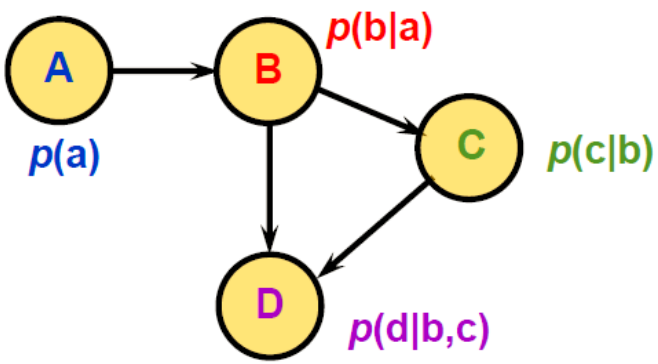
Factorization in BN



$\Rightarrow$ $p(a,b,c,d) = p(a)\,p(b|a)\,p(c|b)\,p(d|b,c)$

- Suppose a, b, c, d are discrete and each have 10 values
- Representing $p(a)$ requires 10 numbers (9, really)
- Representing $p(a|b)$ requires 100 numbers
- Without factorization (no BN): $p(a, b, c, d)$

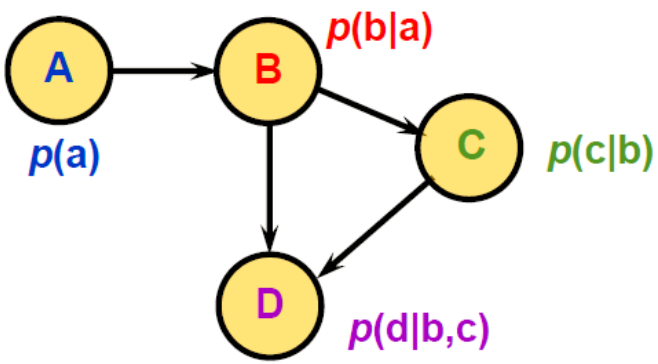
Factorization in BN



$\Rightarrow$ $p(a,b,c,d) = p(a)\,p(b|a)\,p(c|b)\,p(d|b,c)$

- Suppose a, b, c, d are discrete and each have 10 values
- Representing $p(a)$ requires 10 numbers (9, really)
- Representing $p(a|b)$ requires 100 numbers
- Without factorization (no BN): $p(a, b, c, d)$ 10,000
- With factorization:

Factorization in BN



$\Rightarrow$ $p(a,b,c,d) = p(a)\,p(b|a)\,p(c|b)\,p(d|b,c)$

- Suppose a, b, c, d are discrete and each have 10 values
- Representing $p(a)$ requires 10 numbers (9, really)
- Representing $p(a|b)$ requires 100 numbers
- Without factorization (no BN): $p(a, b, c, d)$ 10,000
- With factorization: only $1000 + 100 + 100 + 10 = 1{,}210$